SUP'COM

المدرسة العليا للمواصلات بتونس

ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Cycle de formation des ingénieurs en Télécommunications

Option :

SNI

Rapport de Projet de Fin d'Études

Thème :

« Mise en place d'un pipeline CI/CD pour les applications web multi-services »

Réalisé par :

Fatma Bouhnich

Encadrants :

M. Anouar Bousarsar (DigiCross)

Mme. Sameh Najeh (SUP'COM)

Travail proposé et réalisé en collaboration avec :

DigiCross

Année universitaire : 2020-2021

Ecole Supérieure des Communications de Tunis

SUP'COM

2083 Cité Technologique des Communications - Elghazala - Ariana - Tunisie

Tél. +216 70240 900 – site web : www.supcom.mincom.tn

RÉSUMÉ

L'objectif de ce projet est de mettre en place un pipeline d'intégration continue et de déploiement continu pour les applications Web multiservices. Après une étude des pratiques de DevOps notamment l'intégration continue, le déploiement continue et la gestion de configuration, nous avons effectué une étude comparative des outils de ces pratiques. Suite à cette étude comparative, et avant de mettre en œuvre ce pipeline CI/CD, il est nécessaire de procéder à une analyse des besoins et une conception de l'architecture du système. Ce projet permet de minimiser le temps de déploiement dans les différents environnements et de mieux gérer le cycle de vie des applications, depuis le stockage du code source dans le serveur de gestion des versions, passant par un serveur d'intégration continue et un outil de contrôle qualité jusqu'à son déploiement continu et automatique. Ce projet a été réalisé avec les technologies : Docker, Kubernetes, Ansible, Git, Jenkins, SonarQube, Prometheus, Grafana.

MOTS CLÉS : DevOps, Intégration continue, Livraison continue, Déploiement continu, Automatisation, Virtualisation, Conteneurisation, Docker, ...

Abstract

The objective of this project is to implement a continuous integration, continuous delivery and continuous deployment pipeline for multi-service web applications.
After a study of DevOps practices including continuous integration, continuous deployment and configuration management, we conducted a comparative study of the tools of these practices. Following this comparative study, and before implementing this CI/CD pipeline, it is necessary to conduct a needs analysis and a design of the system architecture. This project allows to minimize the time of deployment in the different environments and to better manage the life cycle of the applications, from the storage of the source code in the version control server, through a continuous integration server and a quality control tool to its continuous and automatic deployment. We brought this solution to light using several technologies such as Docker, Kubernetes, Ansible, Git, Jenkins, SonarQube, Prometheus, Grafana.

KEYWORDS : DevOps, Continuous Integration, Continuous Delivery, Continuous Deployment, Automation, Virtualization, Containerization, Docker,...

Dédicaces

Je dédie ce travail à ma famille qui m'a doté d'une éducation digne, son amour a fait de moi ce que je suis aujourd'hui.

Une dédicace spéciale à la personne qui doit ma vie, ma réussite et tout mon respect : mon cher père. Aucune dédicace ne peut exprimer mon estime, mon amour et mon respect pour vous tout le temps. Cet humble travail est le résultat de tous les sacrifices que vous avez consentis pour ma formation et mon éducation.

À la femme qui ne me laisse pas souffrir, ne refuse jamais ma demande et n'épargne aucun effort pour me rendre heureuse : ma charmante et adorable maman.
Je vous aime mes parents et je supplie le tout-puissant de vous accorder une bonne santé et une vie longue et heureuse.

À mes frères et sœurs qui m'ont toujours soutenu et encouragé durant ces années d'études, Je vous dédie ce travail avec tous mes vœux de joie, de santé et de succès.

À tous mes amis, pour le support et l'aide qu'ils m'ont apportée aux moments délicats, je dédie ce travail avec gratitude et remerciement.

Je dédie ce projet de fin d'études aussi à mes enseignants depuis primaire jusqu'à mon cursus universitaire.

Remerciements

Je voudrais tout d'abord exprimer ma profonde gratitude à toutes les personnes qui ont contribué au succès de mon stage.

J'aimerais remercier M. Anouar Bousarsar et M. Nabil Sghaier de m'avoir accordé l'opportunité d'effectuer ce stage. Particulièrement, Je remercie mon encadrant professionnel M. Anouar pour son encadrement et son suivi. Et pour m'avoir apporté les conseils qui m'ont aidée tout au long de mon stage.

Je tiens à exprimer mes sincères remerciements aussi à mon encadrante interne, Madame Sameh Najeh, pour sa patience, sa disponibilité et surtout ses judicieux conseils.

Que tous ceux et celles qui ont contribué de près ou de loin à accomplissement de ce travail trouvent l'expression de mes remerciements les plus chaleureux et les mots du respect les plus distingués.

Table des matières

Dédicaces **4**

Remerciements **5**

Liste des figures **10**

Liste des abréviations **14**

Introduction générale **15**

1 Cadre général du projet **17**

1.1 Introduction 17

1.2 Entreprise d'accueil 17

1.2.1 Histoire de DigiCross 17

1.2.2 Services de DigiCross 18

1.2.3 Organigramme de DigiCross 18

1.3 Contexte du projet 19

1.3.1 Étude de l'existant 19

1.3.1.1 Analyse de l'existant 19

1.3.1.2 Limites de l'existant 22

1.3.2 Solution proposée 22

1.4 Architecture globale de la solution 23

1.4.1 Git 24

1.4.2 Jenkins 24

1.4.3 SonarQube 24

1.4.4 Ansible 25

1.4.5 Docker 26

1.4.6 Kubernetes 27

1.4.7 Prometheuse 28
1.4.8 Grafana 29
1.5 Conclusion 29

2 État de l'art et étude comparative des outils DevOps 30
2.1 Introduction 30
2.2 Intégration Continue 30
2.2.1 Définition de l'intégration continue 30
2.2.2 But de l'intégration Continue 31
2.2.3 Les étapes de l'intégration continue 31
2.2.4 Les différents composants d'une plateforme d'CI 32
2.2.5 Les avantages de l'intégration continue 33
2.2.6 Étude comparative des serveurs d'intégration continue : Jenkins VS Bamboo 34
2.3 Livraison continue/Déploiement Continu 35
2.3.1 Définition 35
2.3.2 Déploiement Continue vs Livraison Continue 35
2.3.3 Étapes de Déploiement Continu 36
2.3.4 Les avantages du déploiement continu 38
2.3.5 Étude comparative des outils de déploiement continu : Ansible VS Chef VS Puppet 38
2.4 Étude comparative des outils de contrôle qualité 40
2.5 Virtualisation & Conteneurisation 41
2.5.1 Virtualisation 41
2.5.2 Conteneurisation 41
2.5.3 VM VS Conteneurisation 41
2.6 Cloud Computing 43
2.6.1 Définition 43
2.6.2 AWS et ces services utilisés dans notre projet 44
2.7 Conclusion 44

3 Analyse conceptuelle 45
3.1 Introduction 45
3.2 Spécification des besoins 45
3.2.1 Besoins fonctionnels 45

3.2.2 Besoins non fonctionnels 46
3.3 Modélisation conceptuelle de la solution proposée 47
3.3.1 Langage de Modélisation Unifié (UML) 47
3.3.2 Logiciel de Modélisation (StarUML) 47
3.3.3 Diagramme de cas d'utilisation 47
3.3.3.1 Définition 47
3.3.3.2 Diagramme UML 48
3.3.4 Diagramme de composants 49
3.3.4.1 Définition 49
3.3.4.2 Conception avec StarUML 50
3.3.5 Diagramme de déploiement 51
3.3.5.1 Définition 51
3.3.5.2 Diagramme de déploiement UML 51
3.4 Conclusion 52

4 Implémentation et Réalisation de la Solution Proposée 53
4.1 Introduction 53
4.2 Environnement de travail 53
4.3 Implémentaions de la solution 54
4.3.1 Mettre en place de Jenkins 54
4.3.2 Intégration d'un serveur Docker 54
4.3.2.1 Déploiement d'un fichier .gz dans le serveur Docker . 55
4.3.2.2 Configuration de l'environnement docker pour le deployement de l'applications 56
4.3.2.3 Exécution des conteneurs en utilisant docker compose 57
4.3.3 Automatisation de déploiement sur le serveur 58
4.3.4 Intégration de DockerHub dans le pipeline 59
4.3.5 Automatisation de déploiement avec Ansible 60
4.3.6 Création et exécution de l'image docker en utilisant Ansible playbook 61
4.3.7 Job Jenkins pour le déploiement en utilisant Ansible 64
4.3.8 Intégration de cluster Kubernetes dans le pipeline 65
4.3.8.1 La mise en place d'un environnement Kubernetes . . 65
4.3.8.2 La mise en place des nœuds esclaves (slaves nodes) . 66
4.3.8.3 Création des fichiers YAML de déploiement et de service 67

4.3.9 Intégration de Kubernetes avec Ansible 70
4.3.10 CI Job dans Jenkins pour créer une image Docker dans le cluster kubernetes 72
4.3.11 CD Job dans Jenkins pour déployer l'image docker dans Kubernetes 73
4.3.12 Intégration de ESLint avec Jenkins 75
4.3.13 Intégration de SonarQube dans le pipeline 75
4.3.14 Partie monitoring avec Prometheus et Grafana 78
4.3.14.1 Mettre en place d'environnement Prometheus 78
4.3.14.2 Intégration de Prometheus avec Jenkins 79
4.3.14.3 Visualisation & Dashboards 80
4.4 Conclusion 82

Conclusion Générale et Perspectives **84**

Annexes **86**

Liste des références **94**

Table des figures

1.1 Organigramme de DigiCross 18
1.2 Avant DevOps [26] 19
1.3 Après DevOps [26] 20
1.4 Taux de bugs non détécter par le développeur avant et aprés l'CI [16] 21
1.5 Solution DevOps 23
1.6 Logos de Git, Gilab et Github 24
1.7 Intégration d'Ansible dans Devops [6] 26
1.8 Architecture de Docker [2] 27
1.9 Architecture Kubernetes [15] 28

2.1 Concept général de la CI [10] 31
2.2 Composants de la CI 32
2.3 Différence entre Déploiement Continu et Livraison Continue [4] 36
2.4 Étapes de CD [17] 37
2.5 VM vs Conteneur [9] 43

3.1 Diagramme de cas d'utilisation du pipeline 48
3.2 Diagramme de composants du pipeline 50
3.3 Diagramme de déploiement du pipeline 51

4.1 Configuration de Jenkins pour le déploiement de fichier .gz 55
4.2 Authentification avec Jenkins 56
4.3 Dockerfile 57
4.4 Configuration du PollSCM 58
4.5 Lancement du automatiquement job 59
4.6 Répertoire Docker Hub 60
4.7 Modification du compte ansadmin 60
4.8 Publish over SSH 61

4.9 Le premier Playbook 62
4.10 Exécution du premier Playbook 62
4.11 Le deuxième Playbook 63
4.12 Exécution du deuxième Playbook 63
4.13 Caractéristiques du job final 64
4.14 Résultat d'exécution du test final 64
4.15 Installation de kubeadm, kubelet, kubectl et kubernetes-cni 65
4.16 kubeadm init 66
4.17 Afficher les nœuds avec kubectl 66
4.18 Mettre en place le premier nœud 66
4.19 Mettre en place le deuxième nœud 67
4.20 kubectl get nodes 67
4.21 Script YAML de déploiement 68
4.22 Script YAML de service 69
4.23 Vérification après exécution de deploy et service 69
4.24 Fichier Hosts dans /opt/Kubernetes/ 70
4.25 Playbook de déploiement 70
4.26 Exécution du premier playbook 71
4.27 Playbook de service 71
4.28 Exécution du deuxième playbook 71
4.29 CI job Kubernetes 72
4.30 Résultat CI job 73
4.31 Exécution automatique du CD job 73
4.32 CD job k8s 74
4.33 Résultat CD job 74
4.34 Warnings Next Generation Plugin 75
4.35 Installation de SonarQube 75
4.36 Installation de Sonar Scanner 76
4.37 Configuration de sonar scanner dans le serveur Jenkins 76
4.38 Clé d'authentification entre SonarQube et Jenkins 77
4.39 Résultat de l'analyse sonar côté Jenkins 77
4.40 Résultat de l'analyse côté Sonar 78
4.41 Fichier de Configuration Prometheus 78
4.42 Fichier de Configuration Prometheus 79
4.43 Prometheus metrics plugin 79

4.44 Liste de conteneurs 80
4.45 Création d'une source de données du type Prometheus 81
4.46 Dashbord 82
4.47 Connexion SSH 86
4.48 Gitlab Plugin 87
4.49 Génération de paires de clés ssh 88
4.50 Création d'une clé ssh à GitLab 89
4.51 Démarrage de service sonar 90
4.52 Fichier Docker Compose 91

Liste des tableaux

2.1 Tableau comparatif des serveurs d'intégration continue 34
2.2 Tableau des étapes de DC [17] 37
2.3 Tableau comparatif Ansible vs Chef vs Puppet 39
2.4 Tableau comparatif des outils de contrôle qualité 40
2.5 Différence entre VM et conteneur 42

Liste des abréviations

AMI	Amazon Machine Image
API	Application Programming Interface
AWS	Amazon Web Services
CD	Continuous Deployment
CI	Contininous Integration
CPU	Central Processing Unit
DNS	Domain Name System
EC2	Elastic Compute Cloud
HTTP	Hypertext Transfer Protocol
IP	Internet Protocol
IT	Information Technology
K8s	Kubernetes
OS	Operating System
PME	Petites Moyennes Entreprises
RAM	Random Access Memory
REST	Representational State Transfer
SSH	Secure Shell
UML	Unified Modeling Language
URL	Uniform Resource Locator
VCS	Version Control System
VM	Virtual Machine
YAML	Yet Another Markup Language

Introduction générale

La mise en production d'une application est la dernière phase d'un processus bien élaboré faisant intervenir des équipes différentes, principalement équipe de développement et celle des tests. Par conséquent, le développement, les tests et la publication sont assez souvent considérés comme trois phases différentes.
Étant donné que les objectifs de chaque équipe sont différents, le fait d'avoir autant d'équipes impliquées peut entraîner des conflits. Lorsque les développeurs veulent innover et développer les applications, l'équipe de production cherche d'abord à maintenir la stabilité du système informatique. D'ailleurs, chacun suit ses propres processus et utilise ses propres outils pour travailler, et ces outils sont rarement communicants. Les relations entre ces équipes pourraient donc être conflictuelles. Ces conflits peuvent entraîner des retards de livraison, des coûts supplémentaires que l'entreprise n'avait pas prévus au départ, et un impact sur les clients dont la satisfaction du client est au cœur des préoccupations de l'entreprise.
Par conséquent, il est clair qu'une nouvelle méthode est nécessaire pour intégrer les processus de développement et de production afin d'éviter tous les problèmes susmentionnés.

De là est né le concept de DevOps. Il s'agit d'une méthode basée sur la fédération entre la partie exploitation et la partie production. Aujourd'hui, DevOps intervient dès le début du projet. Il élimine cette séparation entre deux équipes appartenant à deux mondes différents, elles ne parlent pas la même langue, si tout doit être livré ensemble pour livrer le produit fini, elles interviendront automatiquement dans le projet. C'est cette rupture qui explique le fait que l'on développe parfois des projets qui ne sortent que quelques mois plus tard. L'alignement de toutes les équipes du système d'informations sur un objectif commun a donc pour effet d'éviter les retards dus à la communication entre ces différents intervenants et de réduire les conflits entre eux, ce qui améliore le délai de livraison.

C'est dans ce contexte que s'inscrit le présent projet au sein de Digicross. Il s'agit de faire la conception et le développement d'une plateforme d'intégration continue et de déploiement continu CI/CD pour les applications web multi-services. En effet, une application web doit passer par son répertoire de gestion du code source, passant par un serveur d'intégration continue et un outil de contrôle qualité jusqu'à son déploiement automatique.

Le plan envisagé dans ce document adopte une démarche progressive et s'articule autour de quatre chapitres se présente comme suit :

1. Dans le premier chapitre, nous ferons une présentation de l'entreprise d'accueil et le cadre général du projet. Et nous exposerons notre solution à mettre en place.
2. Le deuxième chapitre explique les principaux concepts de base du projet, et détaille une étude comparative des outils DevOps les plus répandus que nous avons fait pour argumenter notre choix de technologies.
3. Ensuite le troisième chapitre présente les spécifications des besoins ainsi que l'analyse conceptuelle et modélisation UML que nous faisons avant l'implémentation de notre solution, et se concentre sur le processus de la phase de production.
4. Et enfin dans le quatrième chapitre, nous allons présenter la partie implémentation et réalisation et les différentes tâches de mise en œuvre du projet.

Chapitre 1

Cadre général du projet

1.1 Introduction

Ce chapitre a pour objectif de situer notre travail dans son cadre général. Nous commençons tout d'abord par la présentation de l'entreprise d'accueil DigiCross dans lequel s'est déroulé notre stage, puis nous passons par une étude de l'existant qui permettra de faire une évaluation et critique de ce dernier. Enfin nous présenterons notre solution, ainsi que la démarche de travail suivie.

1.2 Entreprise d'accueil

DigiCross est une Société de Services en ingénierie Informatique spécialisée dans la sous-traitance informatique offshore. DigiCross accompagne leurs clients sur les différentes phases de leurs projets.

1.2.1 Histoire de DigiCross

DigiCross est une startup franco-tunisienne. Sa vision est de dépoussiérer l'image négative dont peut souffrir l'offshore informatique depuis les années 90. Les cofondateurs de DigiCross sont arrivés sur le marché tunisien avec leur propre vision de l'IT et ont développé en 10 ans une stratégie qui place l'humain et le client au cœur des projets. DigiCross est implantée en France en février 2019 et en Tunisie en mai 2019.

1.2.2 Services de DigiCross

L'équipe DigiCross est formée en continu sur les nouvelles architectures et Technologies pour fiabiliser l'ingestion, le stockage et l'exportation de la donnée entreprise. DigiCross vise ainsi la flexibilité des solutions techniques, l'intégrité et le respect des délais. La mission de cette société s'articule sur le développement web et mobile des applications. En effet, les services et l'expérience de DigiCross, à la fois fonctionnels et techniques, sur le Web et les opérations mobiles visent à la création et hébergement de sites Web, développement d'applications d'opérations personnalisées, matérialisent la transformation numérique des PME pour survivre et performer dans une économie numérique [7].

1.2.3 Organigramme de DigiCross

L'organigramme suivant permet de visualiser et déterminer les différentes relations de commandement et les rapports de subordination, d'où une vision simple et claire de la répartition des postes et fonctions au sein de DigiCross.

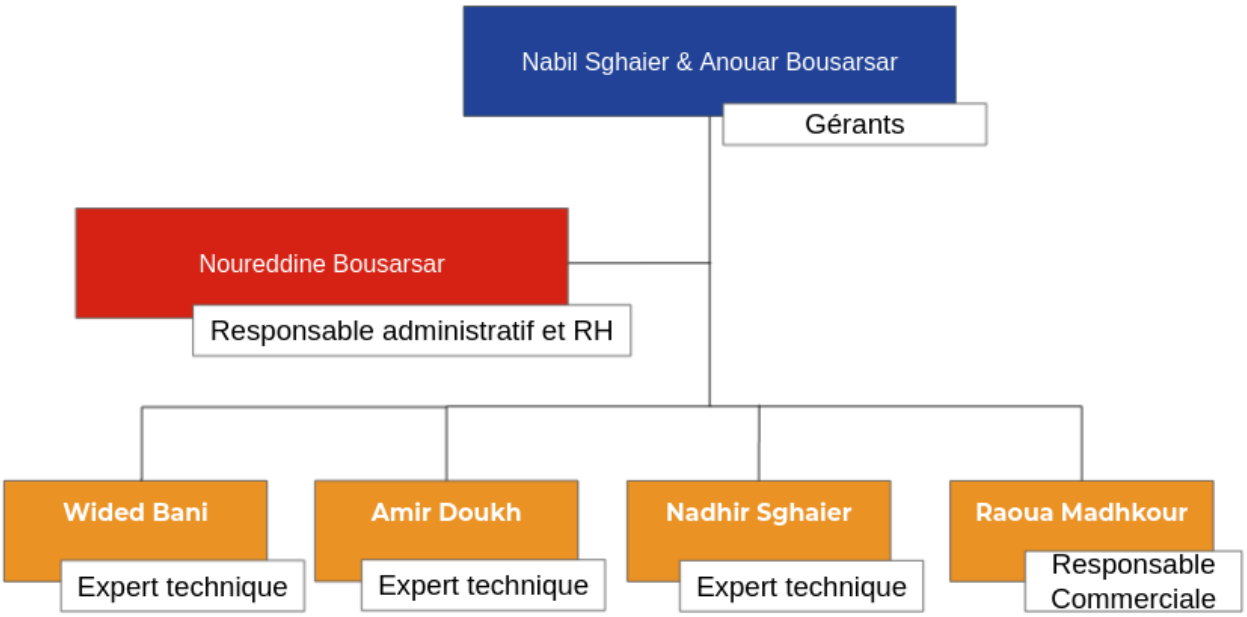


Figure 1.1 – Organigramme de DigiCross

1.3 Contexte du projet

1.3.1 Étude de l'existant

1.3.1.1 Analyse de l'existant

Dans le modèle en cascade traditionnelle, les exigences en matière de logiciel sont claires et prédéfinies à l'avance. La définition du produit même était également stable. Les développeurs sont responsables de faire le codage du logiciel, puis l'équipe d'exploitation doit l'implémenter sur le système de l'entreprise ou sur le Web.

Mais le domaine de l'informatique évolue de plus en plus vite. Ainsi les exigences changent constamment et le développement doit également être plus rapide. Les applications web et les logiciels ne doivent pas uniquement pouvoir être mises plus rapidement sur le marché, mais ils doivent également s'adapter en permanence. Il doit être possible d'ajouter de nouvelles fonctionnalités sans difficulté, et il doit être possible de corriger les erreurs trouvées. Tout ça a conduit au modèle de développement agile.

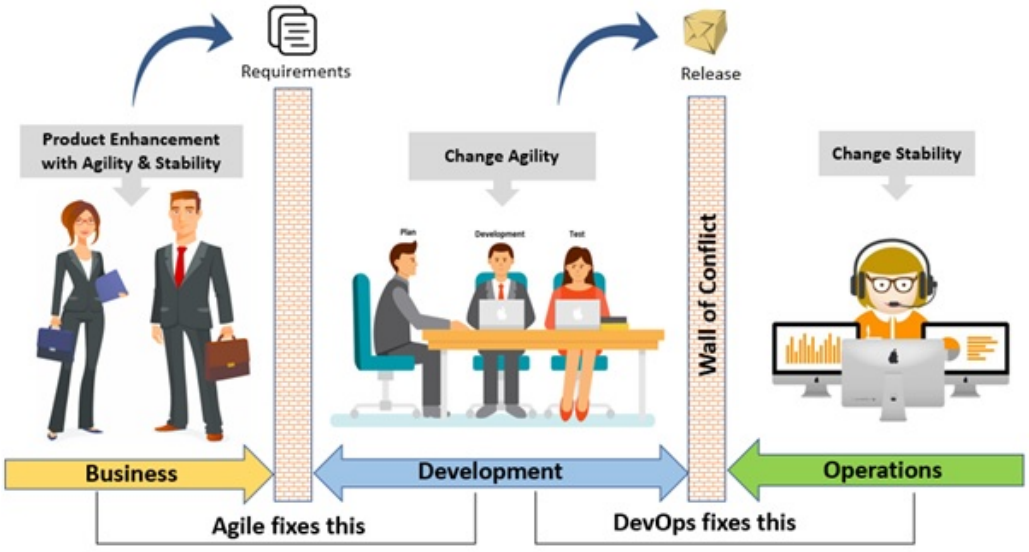


Figure 1.2 – Avant DevOps [26]

Cependant, l'équipe de développeurs n'est pas la seule à devoir réagir de manière rapide et efficace, l'équipe opérationnelle qui doit faire le déploiement et la

surveillance des nouvelles applications, elle aussi doit pouvoir le faire [25].

À ce niveau l'équipe de développement travaille sur du code qui est ensuite envoyé à l'équipe de test pour la validation par rapport aux exigences. L'équipe opérationnelle intervient vers la fin du processus, où le transfert de la libération est donné.

DevOps vise à briser ces silos pour une meilleure collaboration et de meilleures performances.

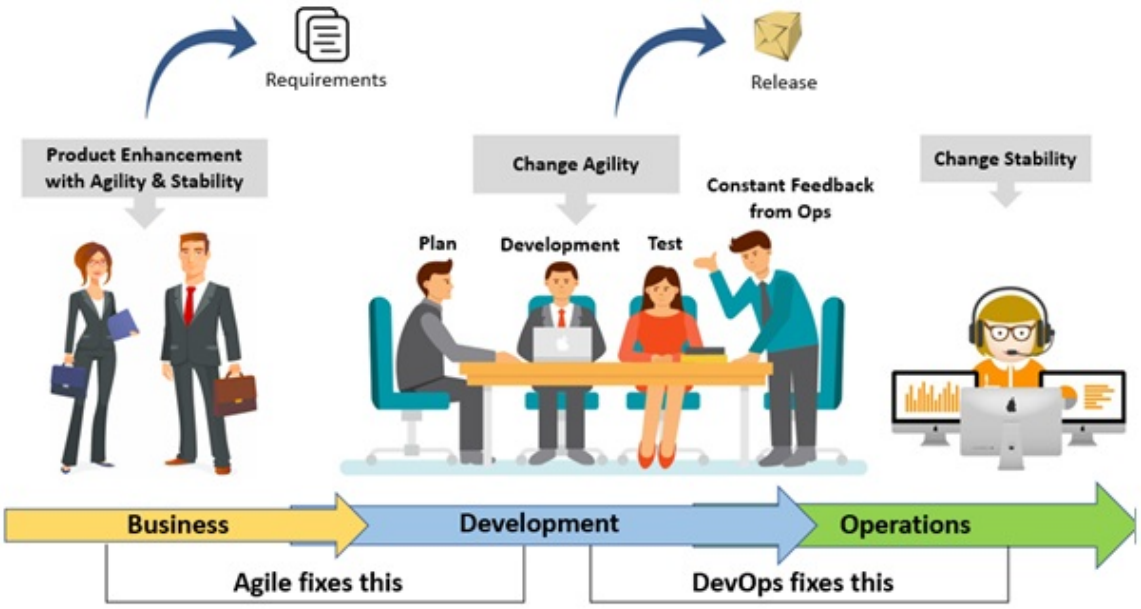


Figure 1.3 – Après DevOps [26]

Grâce à Devops les équipes de développement et d'exploitation travaillent en collaboration pour minimiser les efforts et les risques liés à la publication de logiciels. DevOps affirme qu'une interaction étroite entre l'équipe de développement et l'équipe opérationnelle produira de nouvelles versions du logiciel plus rapidement et avec moins d'erreurs [26].

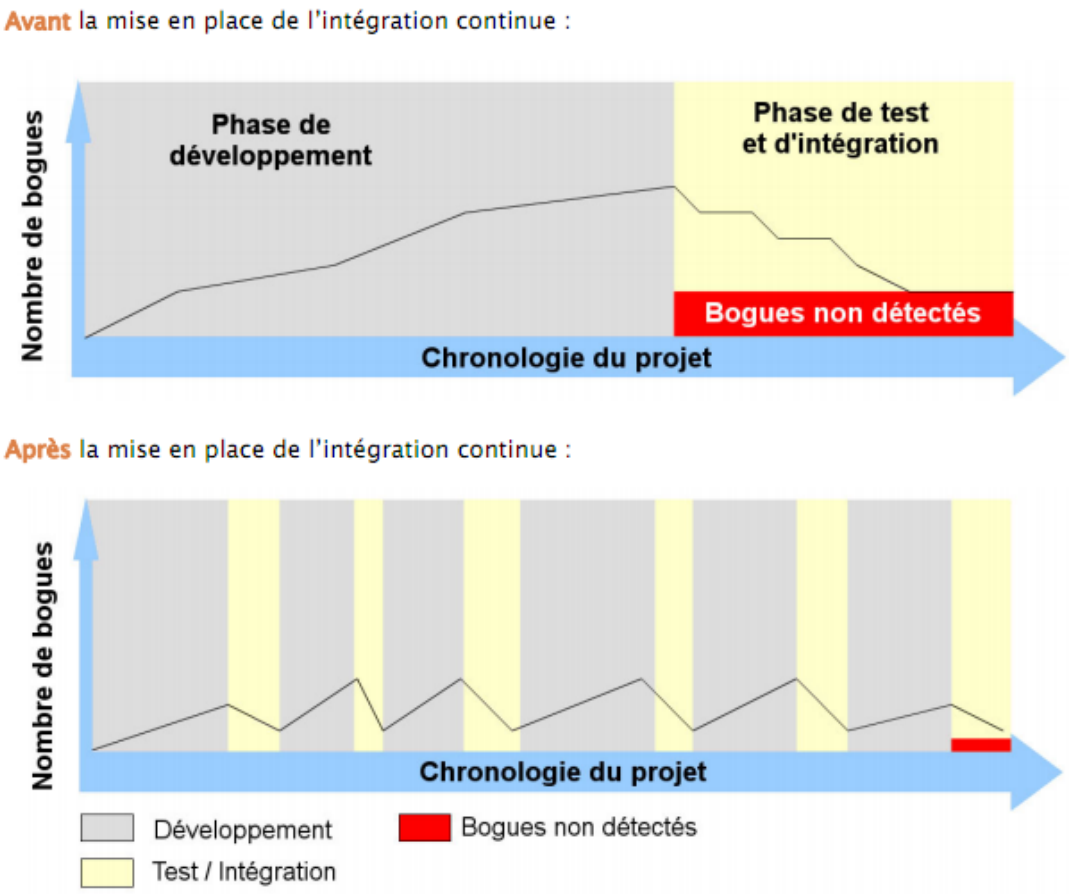


Figure 1.4 – Taux de bugs non détécter par le développeur avant et aprés l'CI [16]

La figure 1.4 nous montre la différence qu'il y a entre un projet qui ne pratique pas l'intégration continue et un autre qui la pratique. Nous voyons que dans les projets qui n'utilisent pas l'intégration continue, il n'y a qu'une seule phase d'intégration. Cette étape est vraiment critique car il n'y a pas de test préalable pendant tout le stade de développement. Alors nous pouvons dire que dans la phase d'intégration, il y a des ambiguïtés qui peuvent poser problème. En contre partie, dans les projets qui utilisent l'intégration continue, nous pouvons voir que plusieurs phases d'intégration sont intégrées dans le processus de développement. Cela permet de configurer de manière incrémentielle plusieurs versions du projet. Ce projet est mené sous forme des itérations de petites tailles. Cette petite taille d'itération permet au producteur de détecter facilement les bugs qui y présente. Donc à la fin du projet nous sommes devant un nombre de bugs beaucoup moins élevé par rapport au premier cas.

Dans le premier cas puisqu'il ya une seule version du projet et il s'agit d'une itération de grande taille, il est donc difficile de détecter toutes les erreurs présentantes dans le projet. Nous pouvons voir donc que plusieurs bugs qui ne sont pas détectés apparaissent lors de stade de test et d'intégration. En mettant en place l'intégration continue, cette phase d'intégration et de tests se répète plusieurs fois et plus souvent, ce qui permet de minimiser cette apparition des bugs.

1.3.1.2 Limites de l'existant

L'analyse de l'existant nous amène à dégager ses limites. Parmi ces limites nous citons :

- Manque de communication entre l'équipe du développement (Dev) et l'équipe opérationnelle chargée de l'exploitation des systèmes (ops).
- Pas des outils d'automatisation pour l'intégration et de déploiement
- Ralentissement des livraisons et des déploiements

Donc toutes ces limites nous poussent à migrer de la méthodologie actuelle vers une méthodologie plus efficace. Nous nous proposons dans le paragraphe suivant de présenter une solution permettant de remédier aux problématiques déjà citées.

1.3.2 Solution proposée

En réponse aux problèmes évoqués dans l'étude de situation actuelle, nous proposons de mettre en place un pipeline d'intégration continue et de déploiement continu CI / CD pour les applications web multi-services.
Avec un pipeline CI / CD, un artefact de version logicielle peut se déplacer et progresser dans le pipeline directement de la phase d'enregistrement du code aux étapes de test, de construction, de déploiement et de production. Ce concept est puissant car une fois qu'un pipeline a été spécifié, certaines parties ou la totalité de celui-ci peuvent être automatisées, ce qui accélère le processus et réduit les erreurs. En d'autres termes, un pipeline CI / CD permet aux entreprises de fournir automatiquement des logiciels plusieurs fois par jour.

CI / CD est une combinaison de deux processus distincts : l'intégration continue et le déploiement continu.

- L'intégration continue (CI) est le processus par lequel le logiciel est construit et les tests initiaux sont effectués. Les modifications de code sont périodiquement intégrées et automatiquement appliquées, testées et prêtes à être déployées.
- Le déploiement continu (CD) est le processus de combinaison du code avec l'infrastructure, en déployant le code dans l'environnement prévu. Cette étape étend le principe de l'intégration continue tous en déployant les modifications de code dans l'environnement de test ainsi que dans l'environnement de production après l'étape de génération (Build) [5].

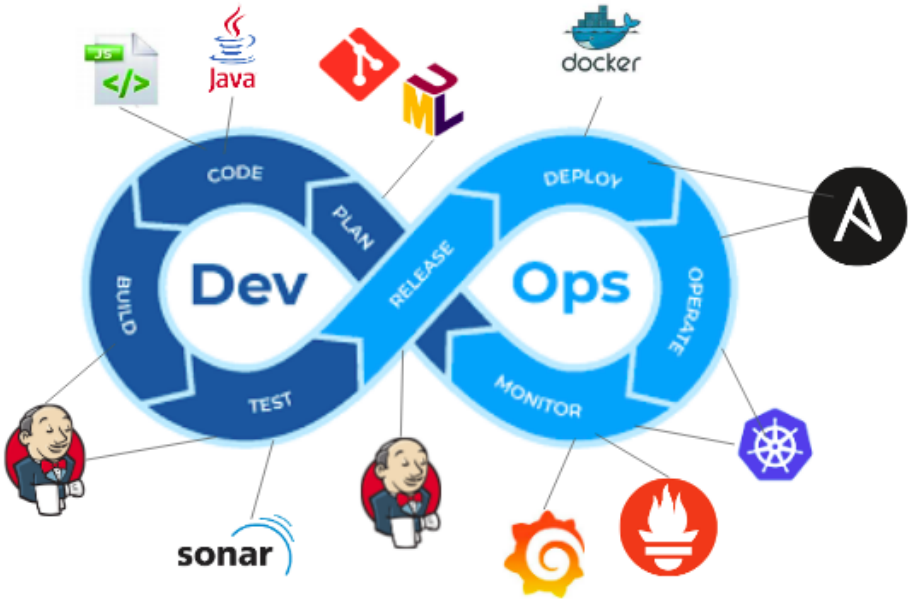


Figure 1.5 – Solution DevOps

1.4 Architecture globale de la solution

La plateforme que nous allons mettre en place est composée de plusieurs outils :

- Un gestionnaire de code source Git
- Un serveur d'intégration continue Jenkins
- Un serveur de déploiement continue Ansible
- Un serveur de contrôle qualité SonarQube
- Un serveur de conteneurisation Docker
- Un serveur d'orchestration Kubernetes
- Un serveur de monitoring Prometheuse

1.4.1 Git

Git est un système de contrôle de versions distribué gratuit et open source son rôle est de gérer tous les projets avec rapidité et efficacité. C'est un logiciel de suivi des modifications dans n'importe quel ensemble de fichiers. Généralement utilisé pour combiner le travail entre les différents programmeurs qui développent le code source en collaboration pendant le développement logiciel [11]. Il existe de nombreuses offres de référentiels Git en tant que service, tel que : GitLab, GitHub, SourceForge, Bitbucket...



Figure 1.6 – Logos de Git, Gilab et Github

1.4.2 Jenkins

Jenkins est un outil open source qui offre un moyen simple de configurer un environnement d'intégration continue et de livraison continue pour presque toutes les combinaisons de référentiels et de langages de code source. Jenkins fonctionne en mode autonome avec son propre serveur Web embarqué ou dans un conteneur de servlets [14].

1.4.3 SonarQube

Les développeurs travaillant avec des délais serrés pour fournir les fonctionnalités requises au client. Il est si important pour les développeurs qu'ils font souvent des

compromis avec la qualité du code, les bugs potentiels, les duplications de code et la mauvaise répartition de la complexité. De plus, ils ont tendance à laisser des variables, des méthodes, etc. inutilisées. Dans ce scénario, le code fonctionnerait de la manière souhaitée.

Pour éviter ces problèmes de code, les développeurs doivent toujours suivre les bonnes pratiques de codage, mais parfois, il n'est pas possible de suivre les règles et de maintenir la bonne qualité car il peut y avoir de nombreuses raisons. Afin de parvenir à une intégration et un déploiement continu du code, les développeurs ont besoin d'un outil qui fonctionne non seulement une fois pour vérifier et leur signaler les problèmes dans le code, mais également pour suivre et contrôler le code afin de vérifier la qualité continue du code. Pour satisfaire toutes ces exigences, il faut mettre en place un outil de contrôle qualité.

Donc pour la partie contrôle qualité du code, nous avons choisi comme serveur SonarQube. En fait SonarQube est une plateforme open source développée par SonarSource pour une inspection continue de la qualité du code. Sonar effectue une analyse de code statique, qui fournit un rapport détaillé des bugs, des vulnérabilités, des duplications de code. Garder le code propre, simple et facile à lire est également beaucoup plus facile avec SonarQube.
SonarQube offre la possibilité non seulement d'afficher la santé d'une application, mais également de mettre en évidence les problèmes nouvellement introduits. Avec Sonar, nous pouvons réparer la fuite et donc améliorer systématiquement la qualité du code [21]. Les tests réalisés par Sonar sont lancés par Jenkins automatiquement à chaque nouveau cycle de vie.
Si la compilation est correctement construite Sonar donne un statut global sur le résultat final de l'analyse : « PASSED » sinon Sonar peut mettre les statuts du cycle en état « FAILURE » si la couverture du code n'atteint pas le pourcentage spécifié. De plus, Sonar doit être lié avec un serveur de base de données (Nous avons utilisé PostgreSQL).

1.4.4 Ansible

Ansible est un moteur d'automatisation informatique radicalement simple qui automatise le provisionnement cloud, la gestion de la configuration, le déploiement des

applications, l'orchestration intra-service [1].

Dans DevOps, comme nous le savons, le travail de développement et d'exploitation est intégré. Cette intégration est très importante pour la conception des applications modernes pilotées par les tests. Par conséquent, Ansible intègre cela en fournissant un environnement stable à la fois au développement et aux opérations, ce qui permet une orchestration fluide.
La figure 1.7 montre comment Ansible s'intègre dans DevOps.

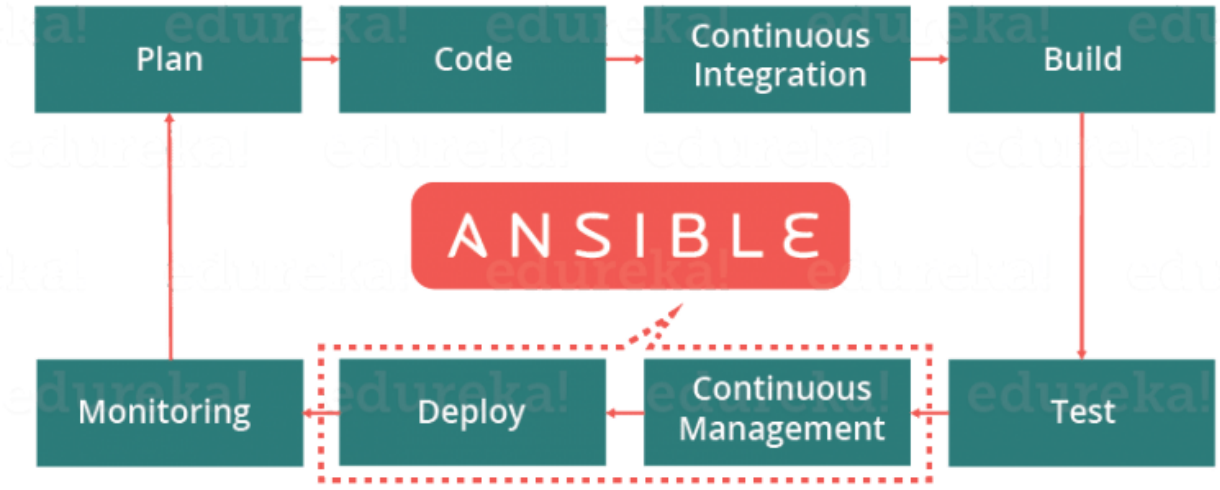


Figure 1.7 – Intégration d'Ansible dans Devops [6]

1.4.5 Docker

Docker est principalement une plate-forme de développement logiciel et une sorte de technologie de virtualisation qui nous permet de développer et de déployer facilement des applications dans des environnements virtuels conteneurisés parfaitement emballés [8].
Les conteneurs Docker peuvent être déployés sur à peu près n'importe quelle machine sans aucun problème de compatibilité afin que notre logiciel reste indépendant du système, cela rend le logiciel plus facile à utiliser, simple à maintenir et à déployer et moins de travail à développer.

En ce qui concerne l'architecture de Docker, elle utilise une architecture client-serveur. Et son architecture se compose de 3 parties principales :

— Hôte Docker
— Client Docker
— Enregistrement

Tout d'abord, le client Docker communique avec le démon Docker, qui s'occupe de la construction, de l'exécution et de la distribution de nos conteneurs Docker. En principe, le client et le démon Docker peuvent fonctionner sur le même système ou nous pouvons connecter un client Docker à un démon Docker distant.
En outre, en utilisant une API REST, le client et le démon Docker communiquent via des sockets UNIX ou une interface réseau.

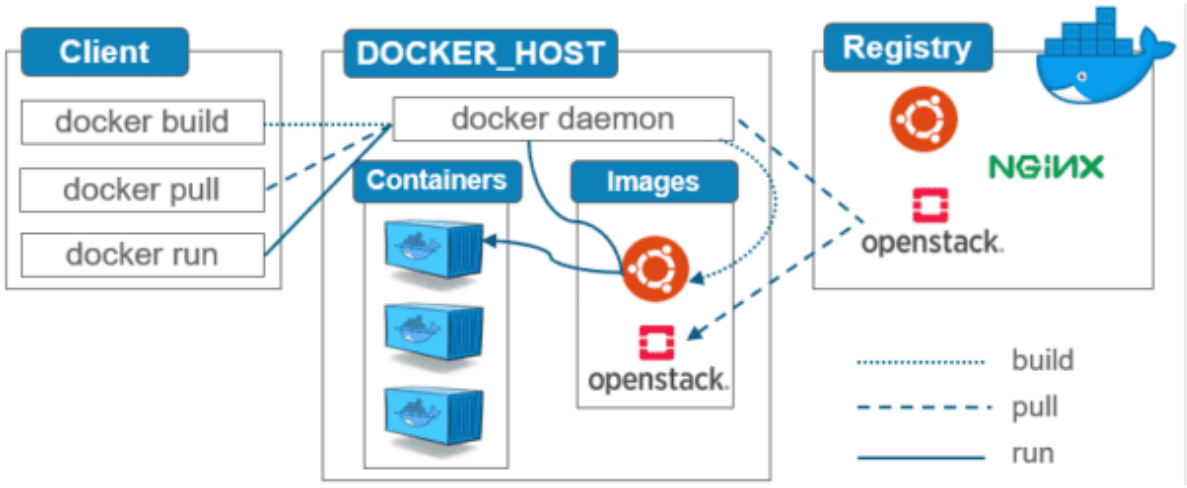


Figure 1.8 – Architecture de Docker [2]

1.4.6 Kubernetes

Kubernetes est un outil d'orchestration open source développé par Google pour gérer des micro-services ou des applications conteneurisées sur un cluster distribué de nœuds. Kubernetes fournit une infrastructure hautement résiliente avec des capacités de déploiement sans temps d'arrêt, une restauration automatique, une mise à l'échelle et une auto-réparation des conteneurs (qui consiste en un placement automatique, un redémarrage automatique, une réplication automatique et une mise à l'échelle des conteneurs en fonction de l'utilisation du processeur) [15].

L'objectif principal de Kubernetes est de masquer la complexité de la gestion d'un parc de conteneurs en fournissant des API REST pour les fonctionnalités requises.

Kubernetes est de nature portable, ce qui signifie qu'il peut fonctionner sur diverses plates-formes de cloud public ou privé telles que AWS, Azure, OpenStack...

Kubernetes suit une architecture maître-esclave. Il est possible d'avoir une configuration multi-maître (pour une haute disponibilité), mais par défaut, il y a un seul serveur maître qui agit comme un nœud de contrôle et un point de contact. Le serveur maître se compose de divers composants, notamment un kube-apiserver, un stockage etcd, un kube-controller-manager, un cloud-controller-manager, un kube-scheduler et un serveur DNS pour les services Kubernetes.
Les composants du nœud incluent kubelet et kube-proxy.

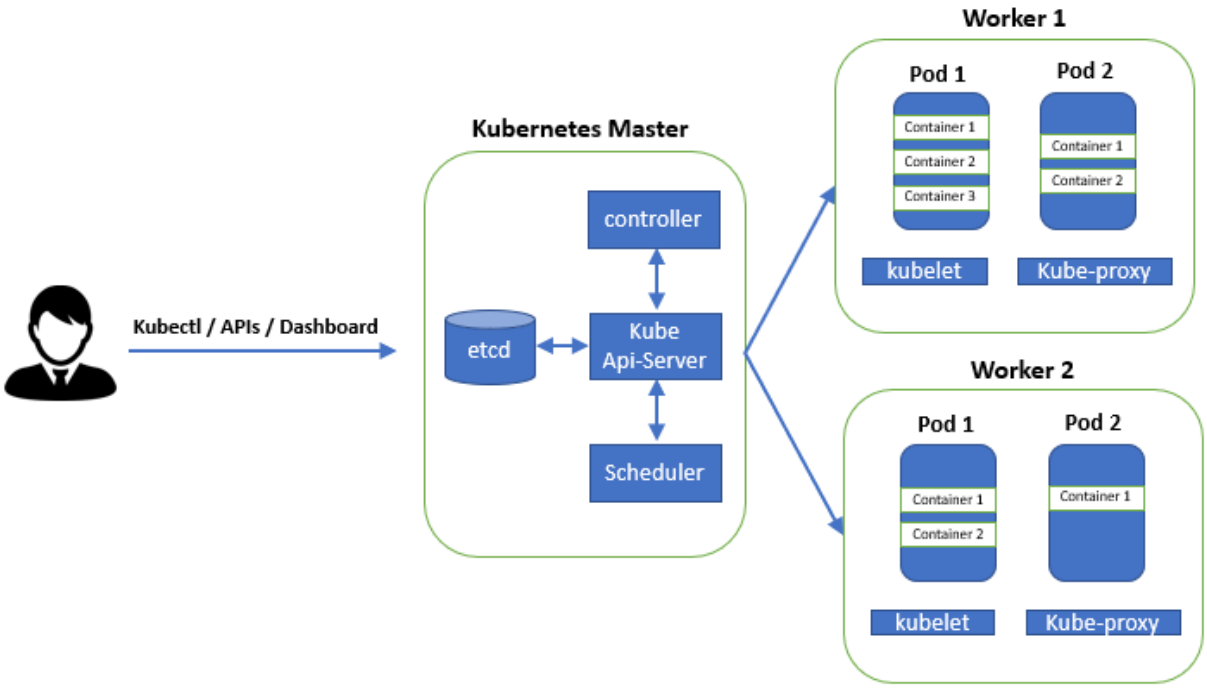


Figure 1.9 – Architecture Kubernetes [15]

1.4.7 Prometheuse

Prometheus a été créé pour surveiller les environnements de conteneurs hautement dynamiques. Au cours des dernières années, prometheus est devenu le choix de surveillance le plus utilisé dans les conteneurs et les micro-services.

Prometheus contribue au système DevOps en surveillant les applications et l'infrastructure natives du cloud et en surveillant des centaines de micro-services. En fait, la surveillance Prometheus peut être utilisée pour clarifier les systèmes et leur fonctionnement [18].

Il stocke tous les échantillons qui sont récupérés localement et il exécute des règles sur ces données pour agréger et enregistrer de nouvelles séries temporelles à partir de données existantes ou générer des alertes.

1.4.8 Grafana

Grafana est une solution open source pour exécuter des analyses de données, extraire des métriques qui donnent un sens à l'énorme quantité de données et pour surveiller nos applications à l'aide de tableaux de bord personnalisables [12].
Grafana se connecte à toutes les sources de données possibles, communément appelées bases de données telles que Graphite, Prometheus, Influx DB, ElasticSearch, MySQL, PostgreSQL, etc. Dans notre projet Grafana est utilisé pour visualiser les données collectées par Prometheus.

1.5 Conclusion

Dans ce premier chapitre, nous avons présenté l'organisme d'accueil DigiCross. Par la suite nous avons étudié la situation avant l'apparition de l'approche DevOps en citant les limites de l'existant, puis nous avons expliqué la solution proposée en présentant son architecture. Puis nous avons cité les différents outils à utiliser dans l'implémentation de notre solution et nous avons défini et précisé l'objectif de chacun d'entre eux. Aussi nous avons expliqué l'architecture Docker et celle de K8s.

Nous détaillerons dans le chapitre suivant clairement les concepts de l'intégration continue et le déploiement continue ainsi que la notion de la virtualisation et la conteneurisation.

Chapitre 2

État de l'art et étude comparative des outils DevOps

2.1 Introduction

Dans ce chapitre nous allons commencer par la mise en évidence des différentes notions de base pour la réalisation de notre projet. Nous aborderons le concept de l'intégration continue. Puis nous définirons la notion de la livraison continue et déploiement continu ainsi que la différence entre eux et nous présenterons ses étapes et ses avantages, suivit par une étude comparative des outils DevOps. Après, nous aborderons la notion de cloud computing. Enfin, nous expliquerons le principe de la virtualisation ainsi que la conteneurisation.

2.2 Intégration Continue

2.2.1 Définition de l'intégration continue

Il s'agit d'un ensemble de pratiques utilisées qui cherche à automatiser les opérations autour du développement ; automatiser l'intégration des modifications de code de plusieurs contributeurs dans un seul projet logiciel. Il s'agit d'une meilleure pratique DevOps principale, permettant aux développeurs de fusionner fréquemment les modifications de code dans un référentiel central où les builds et les tests s'exécutent ensuite. Des outils automatisés sont utilisés pour affirmer l'exactitude du nouveau code avant l'intégration.

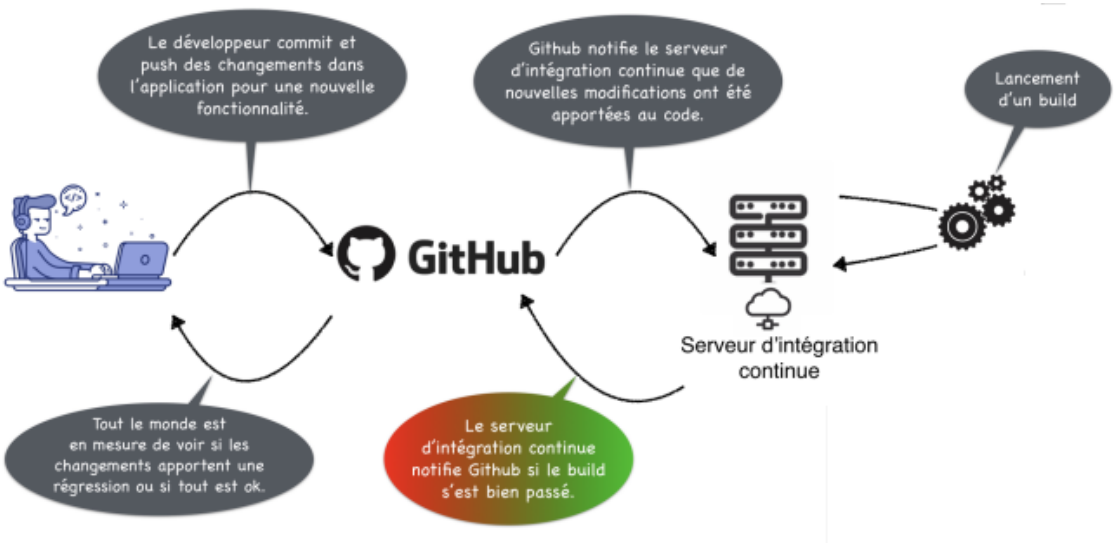


Figure 2.1 – Concept général de la CI [10]

2.2.2 But de l'intégration Continue

Un serveur d'intégration continue doit permettre principalement :

- Détecter les bugs et les problèmes d'intégration le plus tôt possible et les corriger.
- Ne pas attendre la fin du développement du logiciel pour intégrer et corriger un tas de bugs.
- Assurer la qualité de code produit.
- D'effectuer l'opération Checkout du gestionnaire de configuration
- La compilation du code source.
- La création de l'artefact de l'application (Ear, Jar, War,Gz. . .).
- Le déploiement de l'artefact sur la machine de test.
- L'exécution d'une suite de tests.
- La notification du résultat : mail.
- La génération des rapports statistiques.
- L'intégration avec d'autres outils.

2.2.3 Les étapes de l'intégration continue

— Le développeur va procéder à son commit : Après avoir conçu son module, il réalise des tests sur sa machine pour s'assurer le bon fonctionnement dans son

environnement. Il récupère le code pour mettre à jour le code qu'il a sur le gestionnaire de source. Il fusionne le code qu'il a récupéré avec son code puis résout les conflits, et il teste de nouveau son code sur sa machine et apporte des corrections éventuelles. Si tout va bien, alors il publie son code à travers son gestionnaire de code source.

— L'ordonnanceur remarque qu'une nouvelle version est disponible, alors il lance une compilation sur la machine prévue à cet effet.

— Le serveur d'intégration dispose un service de détection de changement de code. Après la publication finale du développeur, il prépare une tâche qui a comme rôle de récupérer la dernière version de code développé et à l'intégrer dans sa plateforme et il exécute cette tâche qui est appelée Job.

— Dès que le job est terminé, des rapports portant sur la stabilité, la qualité ou encore les différents bugs qui peuvent être rencontrés, sont générés et transmis à l'ensemble de l'équipe ou juste au développeur [22].

Une notification est envoyée à l'équipe de développement en cas d'échec. Le développeur concerné par l'erreur fait une mise à jour du référentiel de gestion de configuration et corrige l'anomalie.

2.2.4 Les différents composants d'une plateforme d'CI

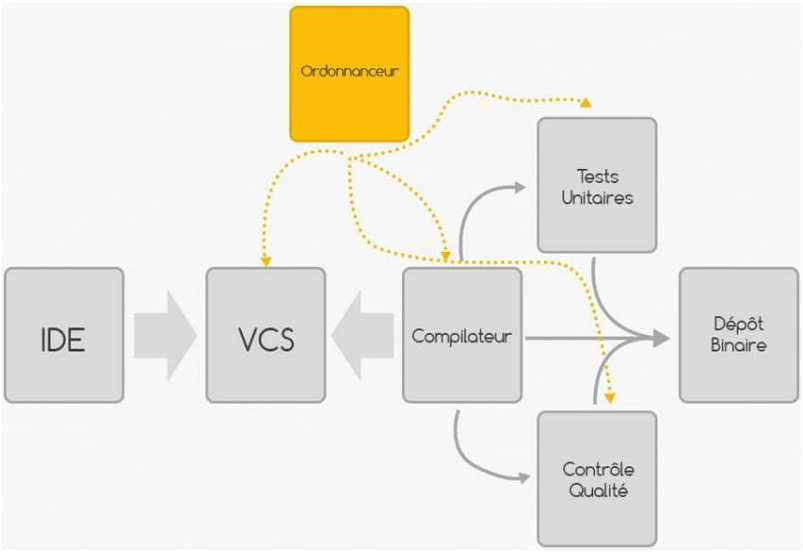


Figure 2.2 – Composants de la CI

L'architecture d'un pipeline d'intégration continue, fait intervenir plusieurs éléments :

— **L'ordonnanceur(serveur moteur de CI) :** C'est un chef d'orchestre du système. Il trie les tâches pour qu'elles soient exécutées dans le bon ordre, et surtout il respecte l'ordre défini par l'administrateur. Il signale également les erreurs à l'utilisateur.
Exemples : Jenkins, Bamboo, ...
— **Le gestionnaire de source (VCS) :** Ici on trouve les différentes versions du code source, sous la forme des répertoires bien organisés selon le choix de l'utilisateur. Tous les développeurs doivent soumettre leur code ici.
— **Le compilateur :** cet outil permet de traduire le code source en langage compréhensible par la machine, ou par un interpréteur.
Exemples : GCC, JDK...
— **Les tests unitaires :** ce composant permet d'effectuer l'ensemble des tests unitaires définis dans un projet.
— **Le contrôle qualité :** ce composant est chargé d'assurer la qualité du code en fonction de plusieurs paramètres.
L'outil le plus classique est SonarQube.
— **Le dépôt de binaire :** il permet de stocker des différentes versions d'un projet après compilation. Il peut aussi servir comme source pour des composants utilisés dans son propre projet.

2.2.5 Les avantages de l'intégration continue

L'intégration continue a des nombreux avantages et qualités, tant en matière de développement qu'en termes d'amélioration d'organisation de projet.

- **Temps :** L'intégration continue permet de détecter et résoudre rapidement les problèmes grâce à l'utilisation des tests unitaires.
- **Argent (Un rapport coût efficacité très intéressant) :** les heures de développement sont généralement des heures facturables et l'automatisation d'une grande partie du flux de travail nous permettra de gagner du temps pour le travail facturable, et donc un gain d'argent.
- **Productivité :** livrer plus rapidement des versions et des mises à jour.

⇒ Avec cette technique d'intégration continue, nous gagnerons en matière de productivité, de temps, et d'argent.

2.2.6 Étude comparative des serveurs d'intégration continue : Jenkins VS Bamboo

Jenkins et Bamboo sont les principaux serveurs d'automatisation avec des plugins conçus pour une intégration continue. Les différences entre Jenkins et Bamboo sont brièvement résumées dans le tableau ci-dessous.

Métrique	**Jenkins**	**Bamboo**
Popularité	Plus populaire que bambou	Moins populaire que Jenkins
Termes de la licence	Open source	Payant
Origine	Langage de programmation Java	Langage de programmation Java
Prise en charge des plugins	Possède plus de 1000 plugins à intégrer à diverses plates-formes	A moins de plugins par rapport à Jenkins
Compatibilité	- Prend en charge les projets de déploiement intégrés - Prend en charge l'API REST - Prend en charge l'automatisation des tests via des plugins - Prend en charge les autorisations de niveau entreprise via des plugins	- Prend en charge les projets de déploiement intégrés - Prend en charge l'API REST - Prend en charge l'automatisation des tests via des plugins - Prend en charge les autorisations de niveau entreprise via des plugins

Table 2.1 – Tableau comparatif des serveurs d'intégration continue

⇒ Donc on se basant sur cette étude comparative nous avons choisi Jenkins comme moteur d'intégration continue puisqu'il est le plus populaire. Aussi Jenkins est l'outil

le plus approprié pour notre cas. De plus ce logiciel est riche en plugins et sa licence est gratuite c'est qui lui donne un pas d'avance par rapport à ses concurrents.

2.3 Livraison continue/Déploiement Continu

2.3.1 Définition

La livraison / déploiement continue est une extension de l'intégration continue. Le déploiement continu est le processus par lequel tout changement du code, y compris les tests automatisés et autres vérifications pertinentes, est automatiquement déployé en production. Le but est de réduire le temps et l'effort du processus de déploiement ainsi que la durée du cycle. Cela permet également aux équipes de développement de réduire le temps nécessaire pour la livraison de fonctionnalités individuelles ou des corrections de bugs, et par conséquent augmente considérablement la productivité des équipes.

2.3.2 Déploiement Continue vs Livraison Continue

— **La livraison continue (CD) :** c'est une pratique d'automatisation de l'ensemble du processus de publication du logiciel. L'idée est de faire CI, ainsi que de préparer et suivre automatiquement une mise en production. Le résultat souhaité est que toute personne disposant de privilèges suffisants pour déployer une nouvelle version peut le faire en quelques clics et à tout moment. Les développeurs deviennent plus productifs en éliminant presque toutes les tâches manuelles.
Le processus de livraison continue comprend généralement au moins une étape manuelle d'approbation et de lancement d'un déploiement en production.

Livraison continue = CI + l'ensemble du processus de publication du logiciel est automatisé (il peut être composé de plusieurs étapes et le déploiement en production est manuel)

— **Le déploiement continu** est une étape par rapport à la livraison continue dans laquelle chaque modification du code source est déployée automatiquement en production, sans l'approbation explicite d'un développeur. Le

déploiement continu nécessite une culture très développée de surveillance, de disponibilité et de capacité de récupération rapide [13].

Déploiement continu = Déploiement entièrement automatisé de CI + livraison continue + Déploiement entièrement automatisé en production

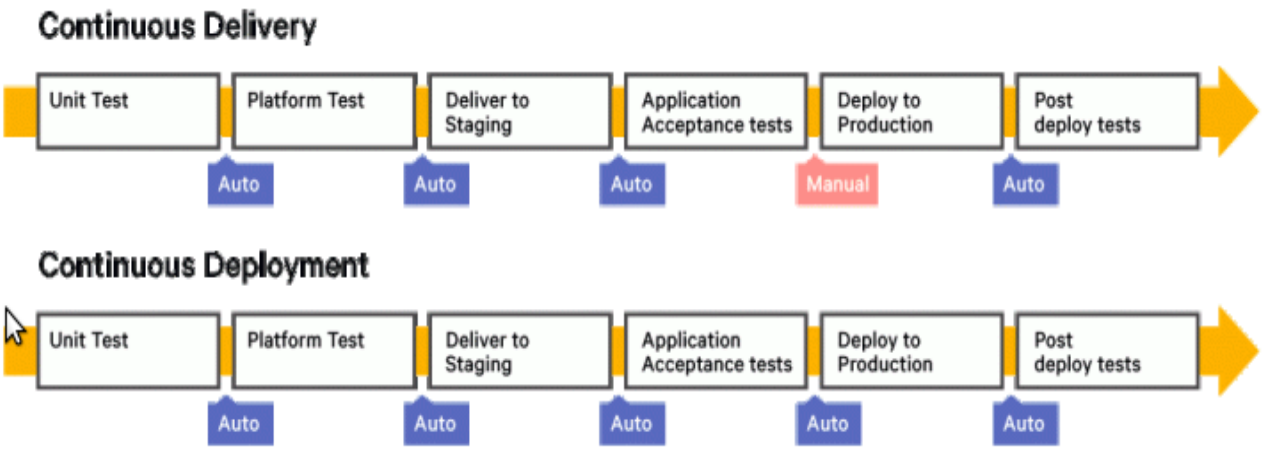


Figure 2.3 – Différence entre Déploiement Continu et Livraison Continue [4]

⇒Comme la montre la figure 2.3 la différence entre le déploiement continu et la livraison continue consiste dans la présence d'un processus manuelle pour mettre à jour et produire : le déploiement en production est manuel dans la livraison continue et il est automatique dans le déploiement continu.

2.3.3 Étapes de Déploiement Continu

La figure2.4 suivante résume les différentes étapes du processus de déploiement continu et qui sont bien expliquées par la suite dans le tableau 2.2 :

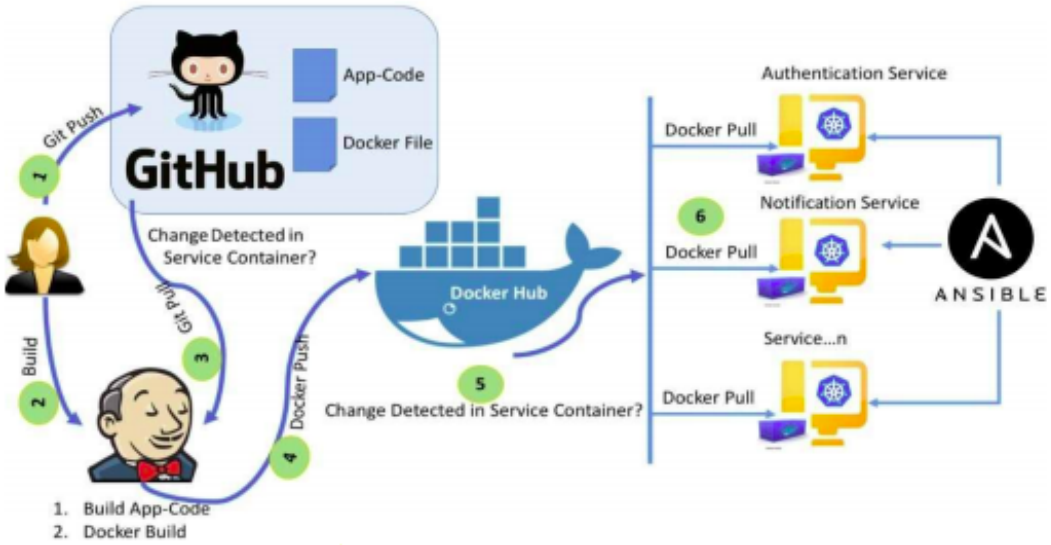


Figure 2.4 – Étapes de CD [17]

Étapes	Description
Commission du code	Le développeur commet son code dans un répertoire de gestion des codes sources (GitHub, GitLab...)
Détection d'un changement	Suite à la commission du code dans le répertoire, le serveur CI remarque un changement et essaye de détecter et d'apporter le changement dans le code. À ce niveau un build est lancé automatiquement.
Tirer le code	le serveur CI tire son code depuis le répertoire
Push vers DockerHub	Le serveur CI, lance un autre build, en donnant l'ordre à un autre serveur d'automatisation tel que Ansible pour faire le push vers un répertoire Docker dans DockerHub.
Serveur d'automatisation continue le déploiement	Après le « push » de l'image, un autre job sera lancé pour terminer la phase de déploiement. Mais ce job ne sera lancé qu'en cas de succès du job précédant du push.
Déploiement vers les serveurs cibles	L'application sera proprement déployée par la suite dans les serveurs cibles qui sont forcément orchestrés par un outil tel que Kubernetes ou bien manuellement, en utilisant la technologie de conteneurisation avec Docker.

Table 2.2 – Tableau des étapes de DC [17]

2.3.4 Les avantages du déploiement continu

Les avantages offerts par le déploiement continu sont les suivants :

— **Automatisation du processus de déploiement logiciel :** La livraison continue permet de créer, tester et préparer automatiquement les modifications de code pour augmenter la vitesse et l'efficacité des projets de livraison de logiciels.

— **Trouver et corriger plus rapidement les erreurs :** Les membres de l'équipe peuvent découvrir et corriger rapidement les erreurs, grâce à des tests plus fréquents et plus poussés, avant qu'ils ne prennent de dimension. Le CD nous permet de mener facilement d'autres types, car tout le processus a été automatisé.

— **Sécurisée :** Les versions sont moins risquées qu'avant la sortie des tests et que tous les bugs sont résolus.

— **Livrer plus rapidement des mises à jour :** La livraison continue aide notre équipe à livrer des mises à jour aux clients plus rapidement et plus fréquemment. Une bonne livraison continue nous permet de toujours suivre un processus de test standardisé, prêt à déployer des artefacts à tout moment.

— **Améliorations continues :** Le déploiement continu prend en charge les améliorations continues visibles pour les clients [19].

2.3.5 Étude comparative des outils de déploiement continu : Ansible VS Chef VS Puppet

Chef, Puppet et Ansible sont des outils DevOps utilisés dans tout le secteur. Ce sont tous des outils de gestion de la configuration, ce qui signifie qu'ils sont conçus pour déployer, configurer et gérer des serveurs.

Métrique	Ansible	Chef	Puppet
Facilité d'installation	Facile	Pas très facile	Pas très facile
Évolutivité	Très évolutif	Très évolutif	Très évolutif
La gestion	**Facile** à apprendre à gérer les configurations car il utilise YAML, c'est-à-dire encore un autre langage de balisage qui ressemble étroitement à l'anglais. Le serveur pousse les configurations vers tous les nœuds. Bon pour les applications en temps réel et il y a une exécution à distance immédiate.	**Pas très facile** : Vous devez être un programmeur pour gérer les configurations car il propose des configurations dans Ruby DSL. Le client extrait les configurations du serveur.	Il n'est **pas très facile** de gérer les configurations car il utilise son propre langage appelé Puppet DSL (Domain Specific Language). Le client extrait les configurations du serveur. Il est plutôt orienté administrateur système et il y a une exécution à distance non immédiate.
Langue de configuration	Ansible utilise YAML (Python). Il est **assez facile** à apprendre et orienter administrateur. Python est intégré dans la plupart des déploiements Unix et Linux de nos jours, donc la configuration de l'outil et son exécution sont plus rapides.	Chef utilise Ruby Domain Specific Language (Ruby DSL). Il a une courbe d'apprentissage abrupte et orienté développeur.	Puppet utilise son propre langage spécifique au domaine (Puppet DSL). Il n'est pas très facile à apprendre et orienter administrateur système.

Table 2.3 – Tableau comparatif Ansible vs Chef vs Puppet

⇒ Après cette comparaison, nous avons choisi de travailler avec Ansible comme serveur de déploiement continu puisqu'il est plus rapide.

2.4 Étude comparative des outils de contrôle qualité

- **SonarQube :** C'est un logiciel libre permettant de mesurer la qualité du code source en continu. Cet outil fournit des rapports sur le code dupliqué, les normes de codage, les tests unitaires, la complexité du code et les failles de sécurité.
- **Scrutinizer :** C'est une plateforme d'inspection continue, aidant les développeurs à créer de meilleurs logiciels. Elle permet de mesurer et surveiller en continu la qualité du code avec des indicateurs de code simples et faciles à comprendre [20].
- **Codacy :** C'est un outil d'analyse de qualité de code qui aide les développeurs à publier rapidement des logiciels de qualité. Cet outil permet d'identifier automatiquement les problèmes grâce à l'analyse de révision du code statique et la surveillance de diverses métriques [3].

Le tableau 2.4 illustre une comparaison des outils de mesure de la qualité des logiciels introduits précédemment :

Aspect	**Sonarqube**	**Scrutinizer**	**Codacy**
Licence	Gratuite	Payante	Payante
Langages supportés	27	7	30
Communauté de support et documentation	Communauté large et documentation claire	Communauté assez pauvre et documentation claire	Communauté pauvre et documentation riche

Table 2.4 – Tableau comparatif des outils de contrôle qualité

⇒ Sur la base de cette étude comparative, nous avons choisi d'utiliser SonarQube comme outil de contrôle qualité. D'une part, en raison de son aspect open source et de sa licence gratuite. D'autre part, pour le grand nombre de langages prises en charge et leur énorme communauté.

2.5 Virtualisation & Conteneurisation

2.5.1 Virtualisation

La virtualisation est une abstraction du matériel à partir du logiciel.
C'est une technique utilisée pour créer et exécuter une ou plusieurs représentations virtuelles d'une machine ou de ses diverses ressources sur une seule machine physique [23].
IL existe 3 grandes formes de virtualisation [24] :

- La virtualisation matérielle : c'est le fait de pouvoir faire tourner plusieurs environnements sur le même serveur physique. Par exemple sur un serveur physique nous allons pouvoir faire tourner deux systèmes d'exploitation.
- La virtualisation de présentation : consiste à exécuter des applications de manière centralisée sur les serveurs en déporter l'affichage sur des postes clients.
- La virtualisation d'application : Elle permet de distribuer des applications métiers sur des postes clients sans perturber le système d'exploitation.

2.5.2 Conteneurisation

La conteneurisation est une technique permettant de virtualiser les applications plutôt que les systèmes d'exploitation. Le conteneur partage le noyau du système d'exploitation hôte avec d'autres conteneurs et la partie partagée du système d'exploitation est en lecture seule. Par conséquent, les conteneurs sont légers, nous pouvons donc déployer plusieurs conteneurs sur un seul serveur (ou une machine virtuelle). Et nous n'avons qu'un seul système d'exploitation à maintenir. La mise à l'échelle devient rapide et facile, sans avoir besoin de plus d'espace sur le serveur.

La conteneurisation résout un certain nombre de problèmes pour le développement et le déploiement de logiciels.

2.5.3 VM VS Conteneurisation

- La virtualisation nous permet d'exécuter plusieurs systèmes d'exploitation sur le matériel d'un seul serveur physique, tandis que la conteneurisation nous permet de déployer plusieurs applications à l'aide du même système d'exploitation sur une seule machine virtuelle ou serveur.

— Les machines virtuelles sont idéales pour prendre en charge les applications qui nécessitent toutes les fonctionnalités d'un système d'exploitation lorsque nous souhaitons déployer plusieurs applications sur un serveur ou lorsque nous avons une grande variété de systèmes d'exploitation à gérer. Les conteneurs sont un meilleur choix lorsque notre plus grande priorité est de minimiser le nombre de serveurs que nous utilisons pour plusieurs applications.
— Dans le cas de la virtualisation l'isolation des VMs se fait au niveau matériel (RAM, CPU, Disque) avec un accès virtuel aux ressources de l'hôte via un hyperviseur. Par contre dans le cas de la conteneurisation, l'isolation est effectuée au niveau du système d'exploitation. Un conteneur s'exécute sous Linux de manière native et partage le noyau de l'hôte avec d'autres conteneurs. Il n'occupe pas plus de mémoire que tout autre fichier exécutable, ce qui le rend léger.

Le tableau 2.5 résume bien les différences entre VM et conteneur.

VM	**Conteneur**
Poids lourd	Poids léger
Performance limitée	Performance native
Chaque machine virtuelle s'exécute dans son propre système d'exploitation	Tous les conteneurs partagent le système d'exploitation hôte
Virtualisation au niveau matériel	Virtualisation du système d'exploitation
Temps de démarrage en minutes	Temps de démarrage en millisecondes
Alloue la mémoire requise	Nécessite moins d'espace mémoire

Table 2.5 – Différence entre VM et conteneur

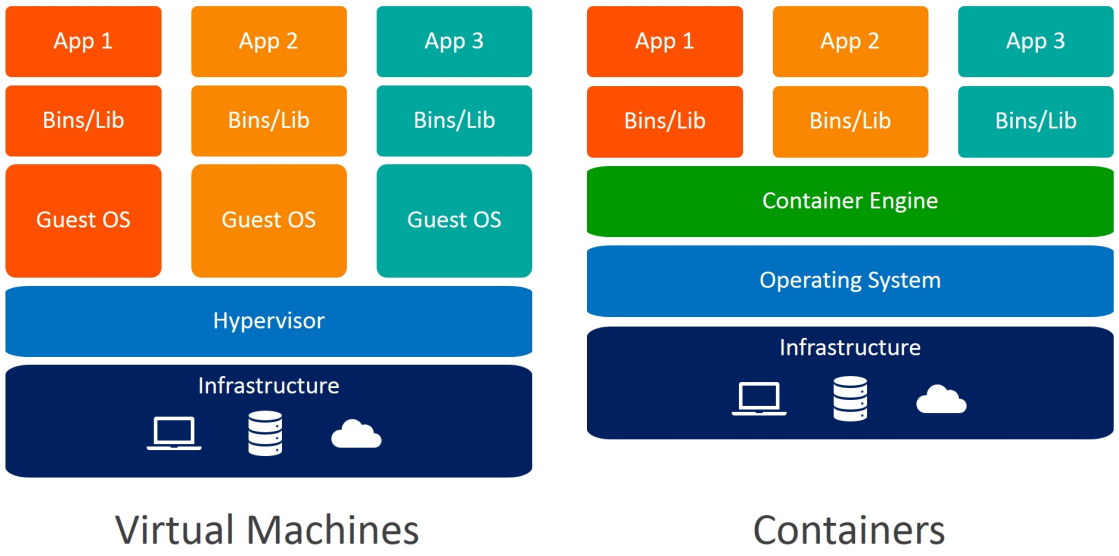


Figure 2.5 – VM vs Conteneur [9]

En se basant sur cette étude comparative nous conclurons que les conteneurs partagent le même système d'exploitation contrairement aux machines virtuelles et que les conteneurs sont beaucoup plus portables et efficaces. Pour cela nous avons choisi la technique de conteneurisation comme solution pour la livraison et le déploiement continu des applications.

2.6 Cloud Computing

2.6.1 Définition

Le cloud computing implique la fourniture de différents types de services sur Internet. Des logiciels et des analyses au stockage de données sécurisé et aux ressources réseau, tout peut être fourni via le cloud pour offrir une innovation plus rapide, des ressources flexibles et des économies d'échelle. En général, nous ne payons que pour les services cloud que nous utilisons, ce qui nous aide à réduire nos coûts d'exploitation, à gérer notre infrastructure plus efficacement et à évoluer en fonction de l'évolution des besoins de notre entreprise.

2.6.2 AWS et ces services utilisés dans notre projet

Amazon Web Services (AWS) fournit des plates-formes de cloud computing et des API à la demande aux particuliers et aux entreprises, sur une base de paiement à l'utilisation. Ces services Web fournissent des nombreuses infrastructures techniques abstraites de base et d'outils informatiques distribués et de blocs de construction. AWS dispose de plusieurs services.

— **Service EC2 :** L'un de ces services est Amazon Elastic Compute Cloud (EC2), qui nous permet de disposer d'un cluster virtuel des serveurs (des instances), disponible tout le temps, via Internet.

— **AMI :** Une AMI (Amazon Machine Image) fournit les informations requises pour lancer une instance. Nous devons spécifier une AMI lorsque nous lançons une instance. Nous pouvons lancer plusieurs instances à partir d'une seule AMI lorsque nous avons besoin de plusieurs instances avec la même configuration. Nous pouvons utiliser différentes AMI pour lancer des instances lorsque nous avons besoin d'instances avec des configurations différentes.

— **RDS :** Amazon Relational Database Service, est un service de base de données relationnelle, son rôle est de simplifier la mise en place, la configuration et le fonctionnement d'une base de données relationnelle à utiliser dans les applications.

2.7 Conclusion

Dans ce chapitre, nous avons présenté les concepts de base de notre projet, nous avons défini le CI et le CD, leurs fonctionnements ainsi que ses composants, et nous avons fait une étude comparative des outils CI/CD pour argumenter notre choix de technologies. Ensuite nous avons défini la virtualisation avec ses différentes formes, et nous avons fini par une étude comparative entre virtualisations ou VM et conteneurisation, et nous avons décrit nos choix concernant les outils de virtualisation.

Avant d'implémenter notre solution nous devons faire une conception et une modélisation de notre travail. Donc le prochain chapitre se concentrera sur l'analyse des besoins de notre plateforme et sur la conception UML.

Chapitre 3

Analyse conceptuelle

3.1 Introduction

Dans ce chapitre nous mettrons en évidence le côté conceptuel de notre projet qui est une étape fondamentale précédente la phase d'implémentation, permet de détailler les différents diagrammes et scénarios à implémenter dans la phase de réalisation mais avant ça nous allons identifier les besoins fonctionnels et non fonctionnels du projet. Ceci permettra de mieux comprendre notre projet. Nous procéderons ensuite à présenter le diagramme de cas d'utilisation. Et enfin nous allons représenter les diagrammes des composants et de déploiement de notre projet qui expliquent les différentes interactions entre chaque composant.

3.2 Spécification des besoins

Dans cette partie, nous analysons d'abord les besoins fondamentaux de notre système à travers la partie besoins fonctionnels, puis les besoins non fonctionnels considérés pour le système.

3.2.1 Besoins fonctionnels

— Le système permet aux développeurs d'exposer ou de récupérer du code à partir du serveur de stockage de code.
— Le système permet de créer des jobs sur Jenkins pour exécuter l'application.

— Le système permet aux développeurs de créer des jobs pour le build et le déploiement des applications.
— Après avoir enregistré le code sur Git, le build sur Jenkins est automatiquement déclenché.
— Le système permet aux développeurs de tester la qualité de code à partir du serveur de contrôle qualité SonarQube.
— Automatiser le déploiement des applications sur le cluster kubernetes.
— Le système permet au client d'accéder à l'application via son URL de production publique.
— Le système permet aux testeurs d'accéder à l'application via son URL de test public pour tester le bon fonctionnement de l'application.
— Le système permet de surveiller l'utilisation du processeur, la RAM et le flux d'entrée/sortie sur le réseau par les pods.
— Le système nous permet de comprendre l'état des différentes applications existantes sur le cluster (container creating, running, pending, etc.).

3.2.2 Besoins non fonctionnels

Les besoins non fonctionnels représentent les règles à respecter afin d'assurer une bonne qualité du processus proposé et le bon fonctionnement du futur système. Ainsi, afin de bénéficier d'une solution fiable et efficace, notre système doit respecter un nombre de critères :

1. **Performance :** Le temps de réponse du système, c'est-à-dire du pipeline et du tableau de monitoring, doit être minimal, au clique sur l'URL.
2. **Convivialité :** L'interface des outils utilisés doit être facile à comprendre et à prendre en main.
3. **Clarté du code :** le code développé doit être clair et bien documenté pour prendre en charge les améliorations futures.
4. **Scalabilité :** Le système doit être capable de supporter l'évolution et l'évolutivité de ses composants, nous pouvons donc ajouter autant de nœuds que nécessaires au cluster kubernetes.

3.3 Modélisation conceptuelle de la solution proposée

Avant l'implémentation de notre solution, et après avoir analysé les besoins fonctionnels et non fonctionnels de notre système, nous devons faire une analyse conceptuelle. Il nous faut tout d'abord définir le langage de modélisation à utilise puis définir le logiciel de modélisation. Nous avons donc opté à UML comme langage de modélisation et StarUML comme logiciel de modélisation.

3.3.1 Langage de Modélisation Unifié (UML)

C'est un langage de modélisation standardisé composé d'un ensemble intégré de diagrammes, développé pour aider les développeurs de systèmes et de logiciels à visualiser, spécifier, construire et documenter les artefacts des systèmes logiciels, ainsi que pour la modélisation d'entreprise et d'autres systèmes non logiciels. L'UML représente un ensemble de bonnes pratiques d'ingénierie qui ont fait leurs preuves dans la modélisation de systèmes importants et complexes.

3.3.2 Logiciel de Modélisation (StarUML)

StarUML est un outil de modélisation logiciel open source qui prend en charge le cadre UML pour la modélisation des systèmes et des logiciels.

3.3.3 Diagramme de cas d'utilisation

3.3.3.1 Définition

Un diagramme de cas d'utilisation est un moyen de résumer les détails d'un système et des utilisateurs de ce système. Il s'agit généralement d'une représentation graphique des interactions entre les différents éléments d'un système. Les diagrammes de cas d'utilisation spécifient les événements dans un système et la manière dont ces événements se déroulent.
Le diagramme général des cas d'utilisation nous permet donc d'exposer les principales fonctions auxquelles le système doit répondre de manière semi-formelle.

3.3.3.2 Diagramme UML

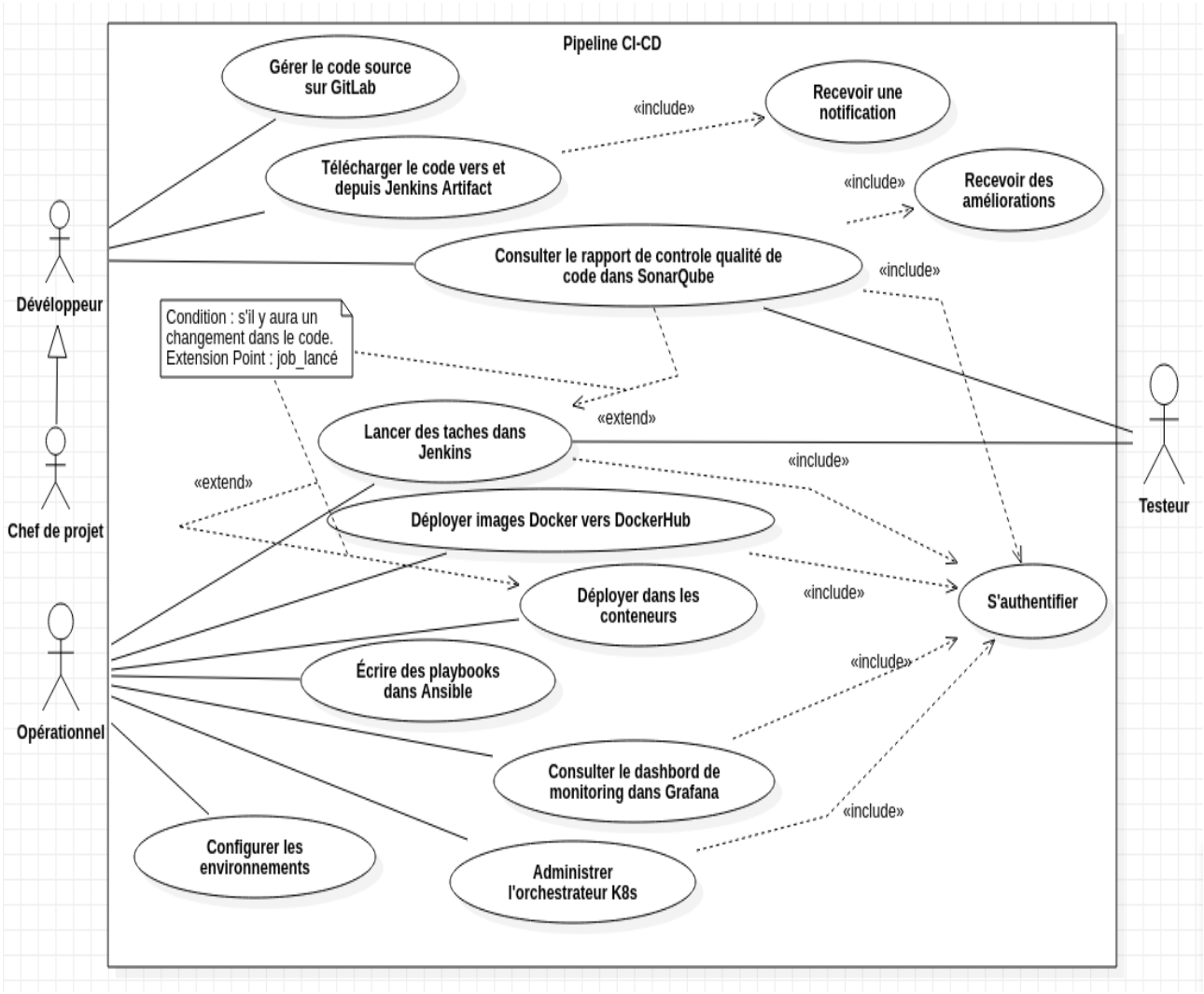


Figure 3.1 – Diagramme de cas d'utilisation du pipeline

Le diagramme des cas d'utilisation général met en évidence les principales fonctions du système. Le développeur peut interagir avec le système de différentes manières. il peut :

— Télécharger le code depuis Jenkins artefacts et le déployer sur un autre serveur après avoir reçu une notification de la part de Jenkins.
— Consulter le rapport de contrôle qualité de code dans le serveur SonarQube et vérifier s'il y a des problèmes ; des warnings, des vulnérabilités, etc. dans

le code, à fin de faire des améliorations au niveau du code. Ça se fait après chaque lancement des taches dans le serveur Jenkins.

D'autre part l'opérationnel peut aussi agir sur le système avec différentes manières. Du coup il peut :

- Lancer des jobs dans Jenkins soit pour l'intégration soit pour le déploiement du code.
- Déployer des images Docker qui ont été créées, dans DockerHub, après avoir fait l'authentification.
- Déployer les images Docker dans des conteneurs d'une manière continue après chaque build des jobs Jenkins.
- Développer des playbooks ansible (scripts) qui assure l'automatisation de déploiement des images Docker.
- Configurer les différents environnements du travail (les serveurs) ; configuration du serveur Jenkins, configuration du cluster kubernetes, configuration serveur de monitoring, configuration du serveur de la qualité de code.
- Consulter le tableau de bord de monitoring dans Grafna pour savoir les états des serveurs.

3.3.4 Diagramme de composants

3.3.4.1 Définition

Un diagramme de composants décrit l'organisation et le câblage des composants physiques d'un système. Des diagrammes de composants sont souvent dessinés pour aider à modéliser les détails de mise en œuvre.

3.3.4.2 Conception avec StarUML

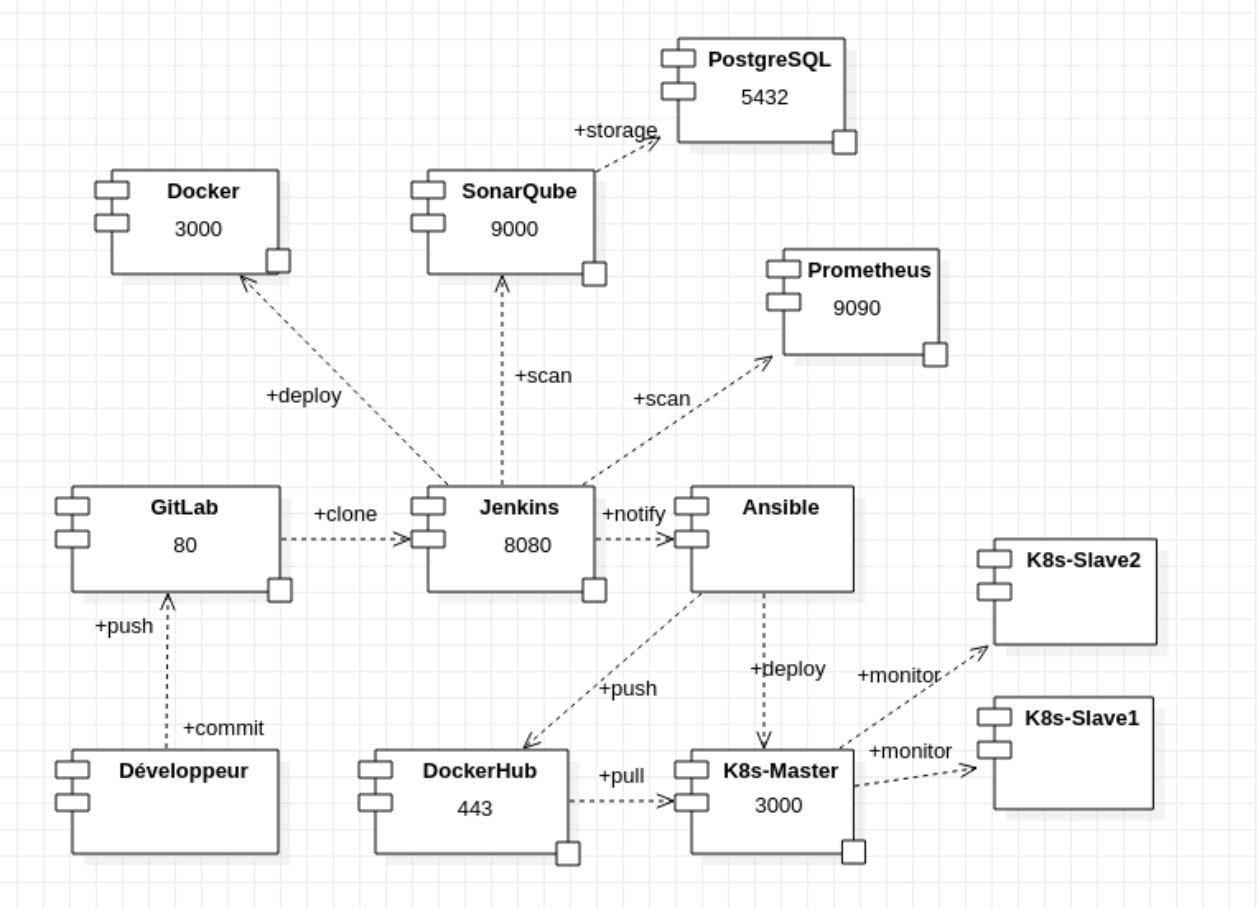


Figure 3.2 – Diagramme de composants du pipeline

Ce diagramme illustre l'enchainement des opérations les plus importantes effectuées par notre pipeline. En fait chaque développeur travaille dans son propre environnement local, pour chaque nouvelle modification du code, il la soumet (fait un commit) au gestionnaire de code source Gitlab. Lorsque nous exécutons un build sous Jenkins, ce dernier télécharge le code source de GitLab, le compile, puis le scanne à l'aide de Sonarqube. En même temps, un autre Job Jenkins sera lancé dans Ansible pour créer une image dans DockerHub et déployer l'image sur un nœud Kubernetes dans un conteneur.

3.3.5 Diagramme de déploiement

3.3.5.1 Définition

Un diagramme de déploiement UML est un diagramme qui montre la configuration des nœuds de traitement d'exécution et les composants qui y vivent. Les diagrammes de déploiement sont une sorte de diagramme de structure utilisé dans la modélisation des aspects physiques d'un système orienté objet. Ils sont souvent utilisés pour modéliser la vue de déploiement statique d'un système.

3.3.5.2 Diagramme de déploiement UML

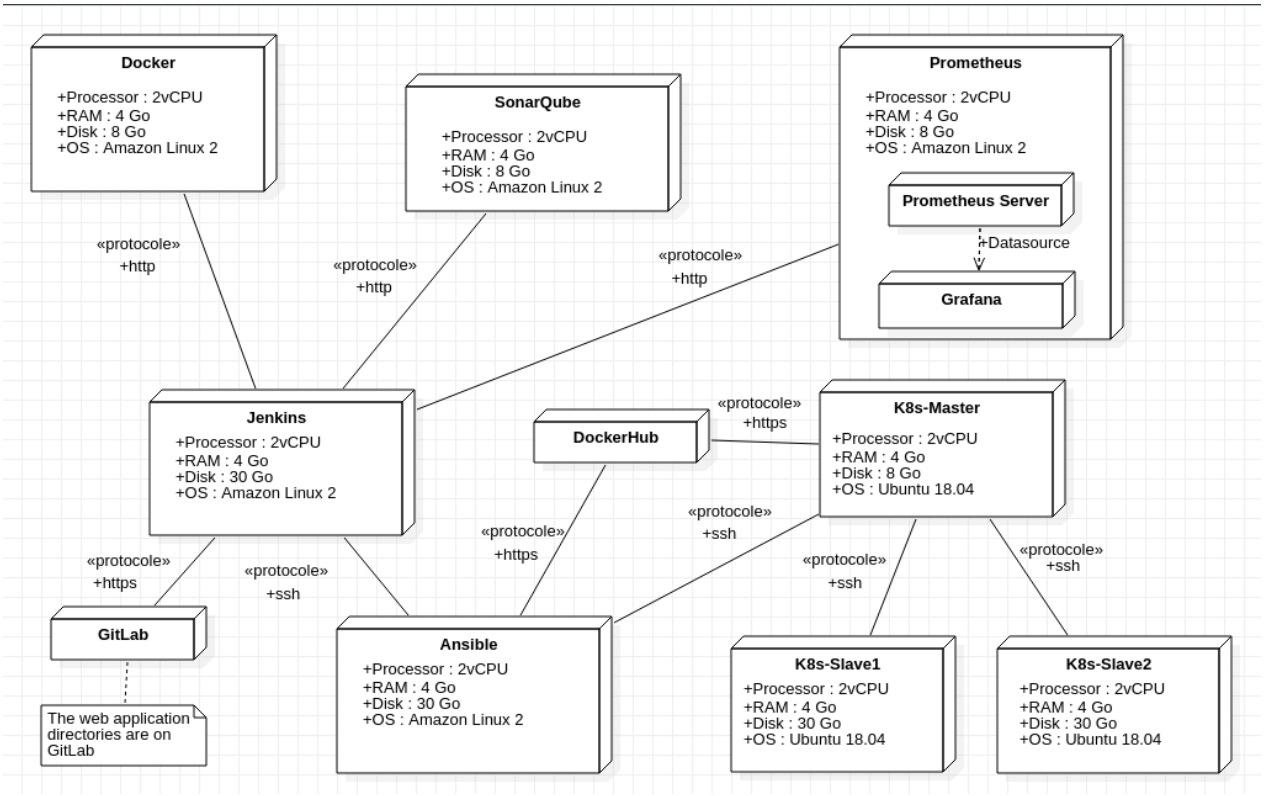


Figure 3.3 – Diagramme de déploiement du pipeline

Le diagramme de la figure 3.3, illustre toutes les étapes de déploiement de la plateforme CI/CD.
Il montre ainsi les détails du déploiement en mettant en évidence les caractéristiques techniques de chaque instance. En fait, ces caractéristiques sont sélectionnées en

fonction des besoins. Ce diagramme met aussi en évidence la communication entre l'instance et les différents protocoles utilisés.

3.4 Conclusion

Dans ce chapitre nous avons entamé les aspects conceptuels du projet avant de commencer la mise en œuvre. Nous avons présenté les besoins de notre projet. Cette représentation est faite avec des diagrammes UML qui illustrent les différents cas d'utilisation et composants du projet, ainsi que le processus de déploiement et les différentes interactions. Nous traitons dans le chapitre suivant la partie implémentation de notre projet.

Chapitre 4

Implémentation et Réalisation de la Solution Proposée

4.1 Introduction

Après avoir traité la partie conceptuelle, nous entamons la partie réalisation de notre projet où nous présentons l'environnement logiciel et matériel de notre travail. Nous allons présenter et expliquer nos travaux techniques pour mettre en place ce pipeline à travers l'essentiel des imprimes écran.

4.2 Environnement de travail

Pour réaliser notre projet, nous avons préparé l'environnement de travail à utiliser qui est composé des éléments suivants :

- Cloud : Amazon Web Service (AWS).
- Services dans le cloud : EC2, AMI, RDS.
- Caractéristiques des serveurs utilisés : Voir le diagramme de déploiement dans le chapitre 3 figure 3.3.

4.3 Implémentaions de la solution

4.3.1 Mettre en place de Jenkins

La première étape dans la construction du pipeline est la création d'un serveur dans lequel nous installons Jenkins. Après la terminaison de l'installation nous pouvons accéder à la page d'accueil Jenkins. Puis, nous exécutons le premier « job » de test qui s'agit d'un seul affichage du message simple pour nous assurer que Jenkins est correctement installé. Et nous exécutons par la suite le fonctionnement de ce « job ». L'étape suivante est l'intégration de Git avec notre serveur Jenkins qui se fait en ajoutons le « plugin » nécessaire (voir annexe).

4.3.2 Intégration d'un serveur Docker

Dans cette étape nous avons lancé une nouvelle instance dans laquelle nous avons installé nodejs et docker. Puis nous avons intégré ce serveur avec le serveur Jenkins. Et après avoir installé les plugins nécessaires dans Jenkins, nous créons un « job » Jenkins qui déploie notre artefact .gz depuis notre répertoire Gitlab, vers le serveur Docker.

4.3.2.1 Déploiement d'un fichier .gz dans le serveur Docker

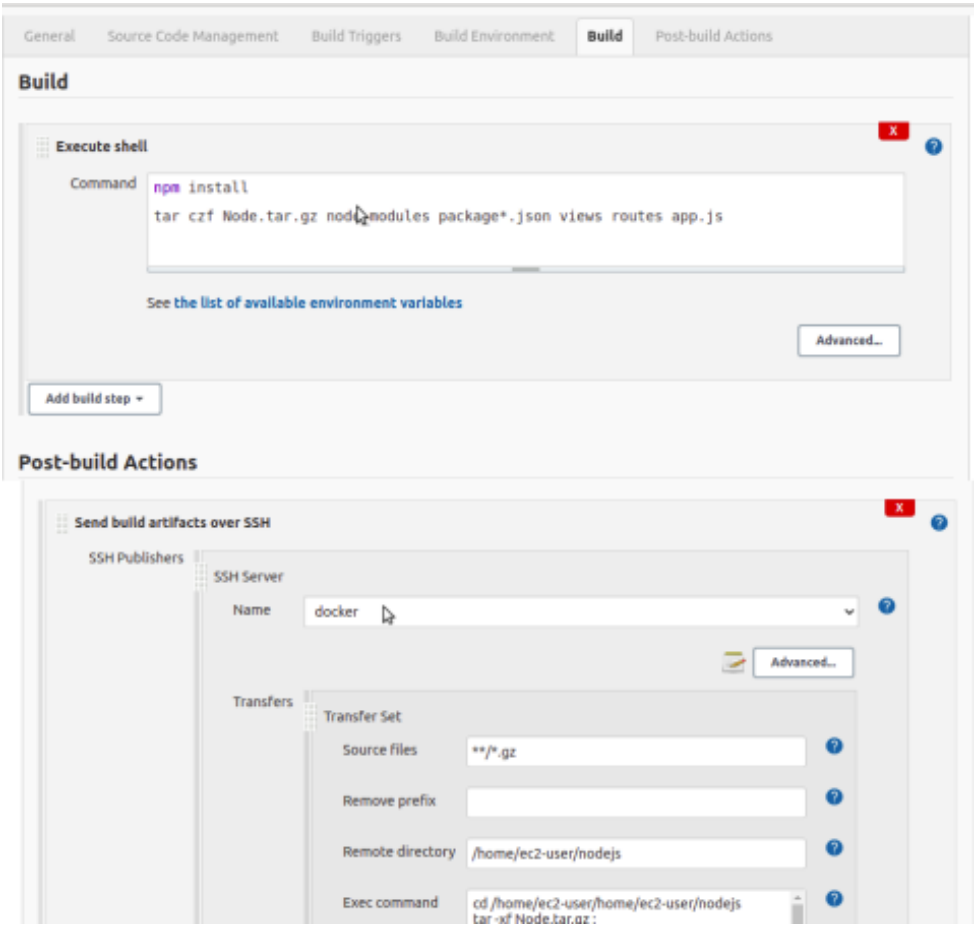


Figure 4.1 – Configuration de Jenkins pour le déploiement de fichier .gz

- Authentification entre Jenkins et le serveur Docker en utilisant le fournisseur d'informations d'identification «ssh username with private key» de jenkins.

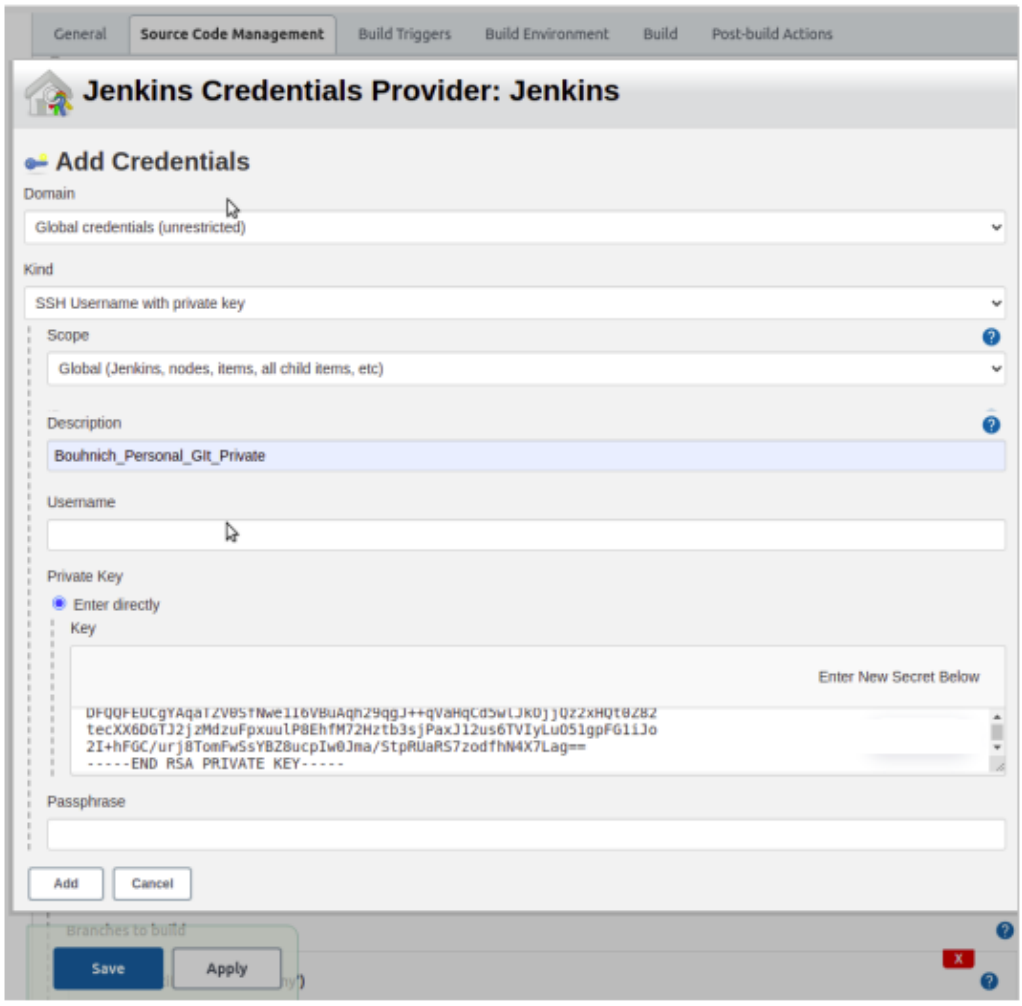


Figure 4.2 – Authentification avec Jenkins

Nous lançons le « job » puis nous vérifions le répertoire dans notre serveur Docker. Nous remarquons que le fichier .gz est déployé dans le dossier du serveur.

4.3.2.2 Configuration de l'environnement docker pour le deployement de l'applications

1. Installation de docker (voir annexe).
2. Création d'un dockerfile pour notre application.

```
[root@docker frontend]# cat dockerfile
# base image
FROM node:10.24.0

# create & set working directory
RUN mkdir -p /usr/src
WORKDIR /usr/src

# copy source files
COPY . /usr/src

# install dependencies
RUN npm install

EXPOSE 3000
CMD [ "npm", "run", "dev" ]
[root@docker frontend]#
```

Figure 4.3 – Dockerfile

3. Création de trois conteneurs (Un pour la partie frontend et un autre pour la partie backend et un troisième conteneur pour la base de données) pour notre application en utilisant des commandes docker.

4.3.2.3 Exécution des conteneurs en utilisant docker compose

Puisque nous avons un ensemble composé de 3 conteneurs et pour éviter l'exécution de chaque conteneur dans une ligne de commande à part, nous avons utilisé docker compose qui permet de gérer plusieurs conteneurs comme un ensemble de services interconnectés.

Nous pourrons alors démarrer notre ensemble de conteneurs en une seule commande **docker-compose up**.

4.3.3 Automatisation de déploiement sur le serveur

Pour l'automatisation de déploiement sur le serveur il y a 2 possibilités soit nous utilisons la technique PollSCM soit nous utilisons les web hooks. En fait les deux techniques ont le même rôle qui est interroger périodiquement pour vérifier si des modifications ont été apportées dans l'application. En effet, PollSCM est une opération lourde et dépendre de lui pour déclencher la construction signifie que nous devons l'activer fréquemment. Mais la configuration est plus simple et plus sûre que les web hooks car Jenkins communiquera avec le dépôt de code.
Les web hooks peuvent nous donner l'heure exacte du déclenchement de la construction sans avoir à la vérifier constamment. Mais il y a des problèmes de sécurité lorsque nous ouvrons une connexion de l'extérieur et la configuration n'est pas facile par rapport à PollSCM.
Donc si le cycle de construction est très court (quelques minutes) et si nous voulons déclencher une construction pour chaque livraison, la solution des Webhooks est meilleure. Mais si le cycle de construction est plus long et si nous n'avons pas besoin de construire pour chaque commit, PollSCM est un bon candidat.
Donc dans notre cas nous avons utilisé la technique PollSCM.

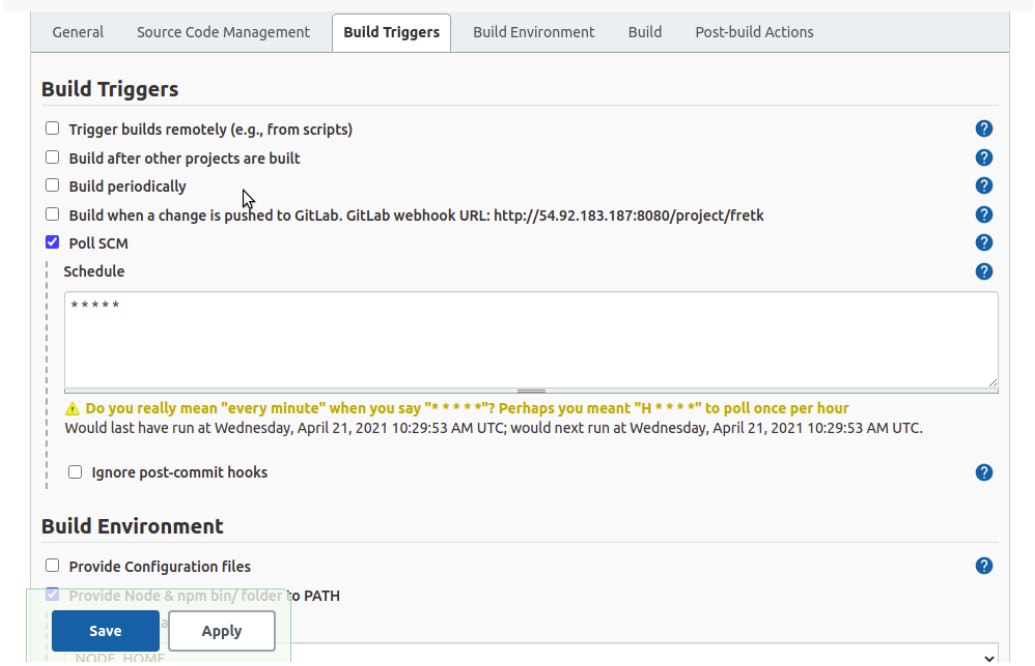


Figure 4.4 – Configuration du PollSCM

Donc à ce niveau si nous effectuons un changement au niveau du code et nous faisons le « push » vers la répertoire dans Gitlab, nous remarquons à ce moment qu'un build du « job » Jenkins est lancé **automatiquement**, ce qui signifie que Jenkins détecte le changement effectué au niveau de gitlab. Et la figure suivante illustre cet événement :

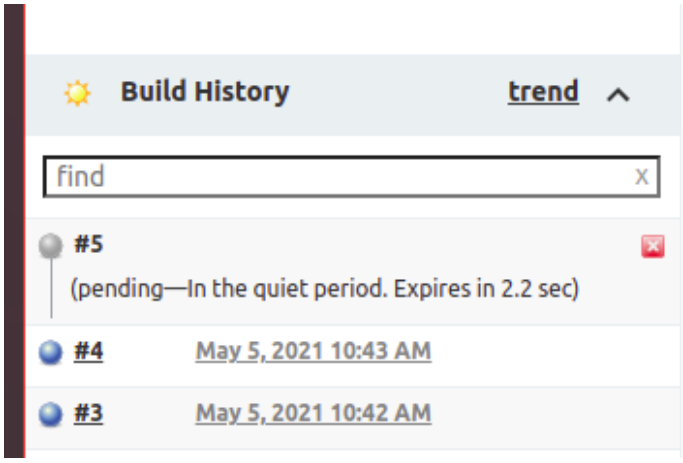


Figure 4.5 – Lancement du automatiquement job

Nous vérifions l'exécution de notre application en visitant, dans notre navigateur, l'adresse IP public de notre serveur sur le port 3000.

4.3.4 Intégration de DockerHub dans le pipeline

Pour enregistrer et stocker nos images docker et pour créer des conteneurs dans le serveur docker il faut intégrer Dockerhub dans notre pipeline. Donc pour faire ceci il faut tout d'abord créer un compte sur Dockerhub, puis créer un répertoire dans lequel nous pouvons pousser nos images. Nous allons faire le « tag » de notre image avant de la pousser vers Docker Hub.

Vérification dans Docker Hub :

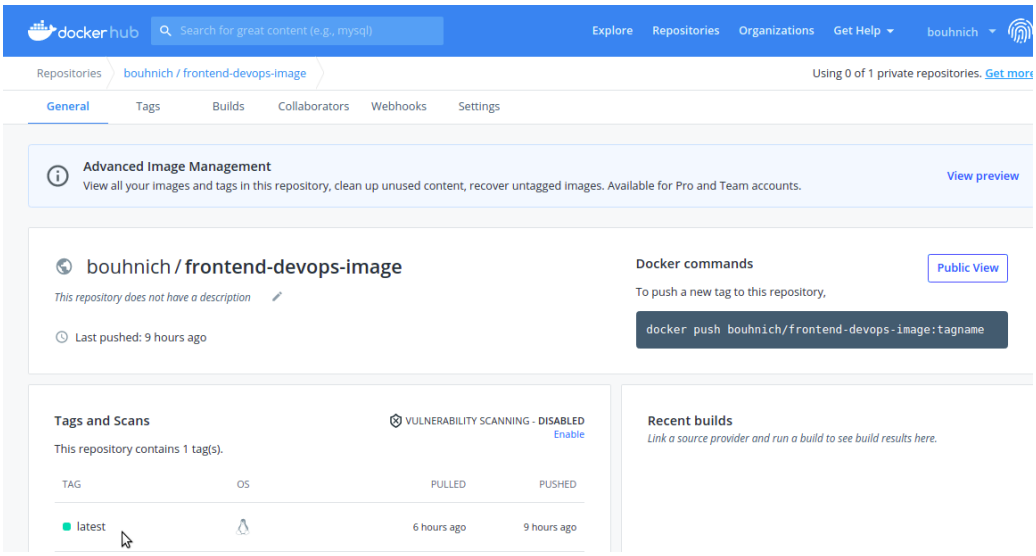


Figure 4.6 – Répertoire Docker Hub

Enfin, nous supprimons l'image localement, pour que la prochaine fois, nous ne tirons l'image que depuis Docker hub.

4.3.5 Automatisation de déploiement avec Ansible

Pour cette partie nous allons suivre les étapes suivantes :

- La mise en place d'un environnement Ansible :
  - Création d'un serveur Ansible_server sur aws
  - Installation d'Ansible mais avant ça il faut installer Python
- Ensuite nous ajoutons ansadmin un utilisateur de serveur ansible et nous éliminons l'exigence d'un mot de passe.
- Puis nous modifions le compte utilisateur ansadmin et autorisons l'accès par mot de passe au service AWS EC2.

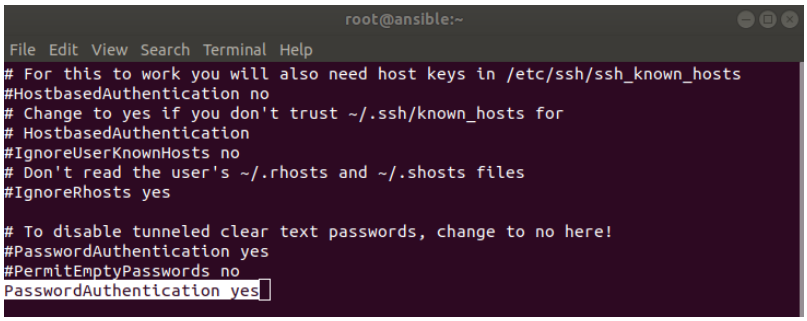


Figure 4.7 – Modification du compte ansadmin

— Nous créons un fichier hosts contenant les adresses IP qu'Ansible adressera et nous utiliserons «ping» pour tester la connectivité avec le serveur.

— Installation du plugin « Publish over SSH » à travers lequel nous pouvons se connecter avec le serveur Jenkins tout en configurant la section « SSH Server » et en remplissant les champs qui manquent comme l'indique la figure ci-dessous :

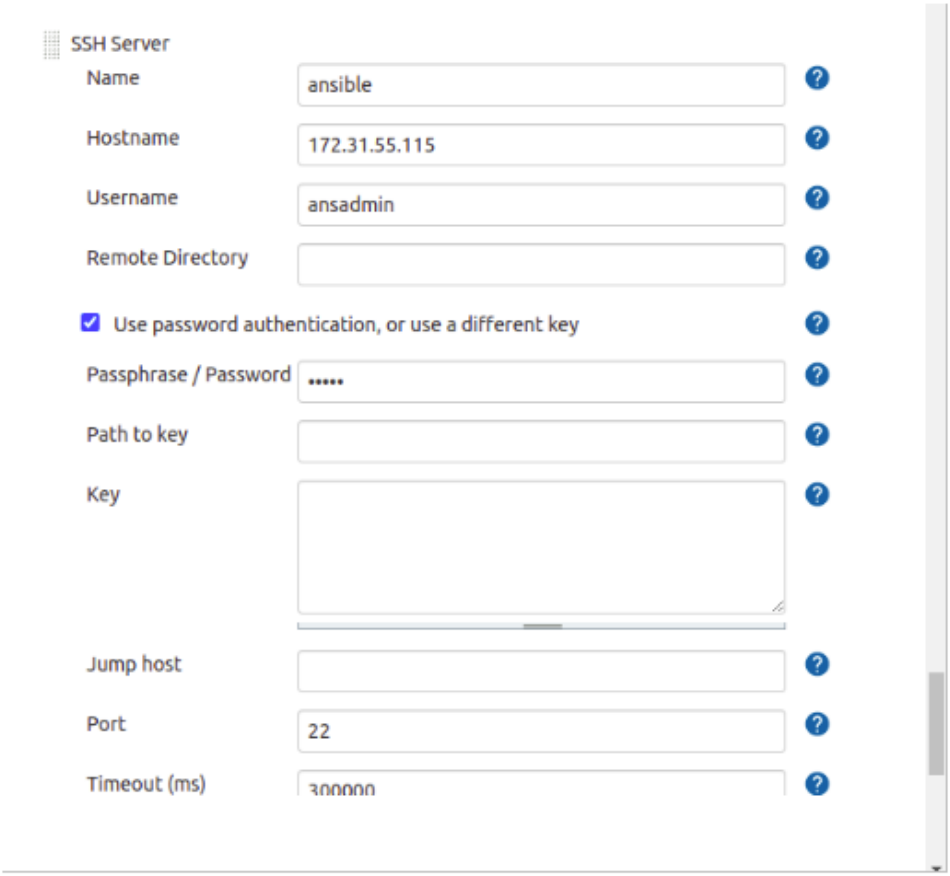


Figure 4.8 – Publish over SSH

4.3.6 Création et exécution de l'image docker en utilisant Ansible playbook

À ce niveau, nous allons créer deux playbooks :

1. Un playbook pour le build de l'image docker et sa mise en pace dans docker Hub, ce playbook doit être écrit dans le serveur Ansible.

2. Et un autre playbook dans le serveur Docker qui assure le pull de l'image docker à partir de dockerHub.

```
[ec2-user@Ansible docker]$ cat create-frontend-devops-image.yml
---
- hosts: all

  tasks:
  - name: create docker image using gz file
    command: docker build -t frontend-devops-image:latest .
    args:
      chdir: /opt/docker/front

  - name: create tag to image
    command: docker tag frontend-devops-image bouhnich/frontend-devops-image

  - name: push image to dockerhub
    command: docker push bouhnich/frontend-devops-image

  - name: remove docker image from ansible server
    command: docker rmi frontend-devops-image:latest bouhnich/frontend-devops-image
    ignore_errors: yes
[ec2-user@Ansible docker]$
```

Figure 4.9 – Le premier Playbook

Exécution du premier Playbook :

```
[ansadmin@ansible front]$ ansible-playbook -i hosts create-devops-image.yml

PLAY [all] *********************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python, but future installation of another
Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible/2.10/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [create docker image using gz file] ***************************************************************************************************************
changed: [localhost]

TASK [create tag to image] *****************************************************************************************************************************
changed: [localhost]

TASK [push image to dockerhub] *************************************************************************************************************************
changed: [localhost]

TASK [remove docker images from ansible server] ********************************************************************************************************
changed: [localhost]

PLAY RECAP *********************************************************************************************************************************************
localhost                  : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ansadmin@ansible front]$
```

Figure 4.10 – Exécution du premier Playbook

Le deuxième Playbook :

```
[ec2-user@Ansible docker]$ cat devops-project.yml
---
- hosts: all

  tasks:
   - name: stop current running container
     command: docker stop FRONTEND_CONTAINER
     ignore_errors: yes

   - name: remove stopped container
     command: docker rm FRONTEND_CONTAINER
     ignore_errors: yes

   - name: remove docker image
     command: docker rmi bouhnich/frontend-devops-image
     ignore_errors: yes

   - name: remove docker image
     command: docker rmi frontend
     ignore_errors: yes

# - name: build docker image
#   command: docker build -t frontend .
#   args:
#     chdir: /opt/docker/front


   - name: pull docker image from dockerhub
     command: docker pull bouhnich/frontend-devops-image:latest

   - name: create container using devops image
     command: docker run --name FRONTEND_CONTAINER -p 0.0.0.0:3000:3000 bouhnich/frontend-devops-image

[ec2-user@Ansible docker]$
```

Figure 4.11 – Le deuxième Playbook

Exécution du deuxième playbook :

```
[ansadmin@Ansible front]$ vi fretk-devops-project.yml
[ansadmin@Ansible front]$ ansible-playbook -i hosts fretk-devops-project.yml

PLAY [all] *********************************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python, but future installation of another
Python interpreter could change the meaning of that path. See
https://docs.ansible.com/ansible/2.10/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [stop current running container] ******************************************************************************************************************
changed: [localhost]

TASK [remove stopped container] ************************************************************************************************************************
changed: [localhost]

TASK [remove docker image] *****************************************************************************************************************************
changed: [localhost]

TASK [pull docker image from dockerhub] ****************************************************************************************************************
changed: [localhost]

PLAY RECAP *********************************************************************************************************************************************
localhost                  : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ansadmin@Ansible front]$
```

Figure 4.12 – Exécution du deuxième Playbook

Remarque : Nous répétons les mêmes étapes avec l'image de la partie backend.

4.3.7 Job Jenkins pour le déploiement en utilisant Ansible

Dans cette étape nous allons créer un nouveau « job » Jenkins pour exécuter les playbooks d'ansible à partir de Jenkins.

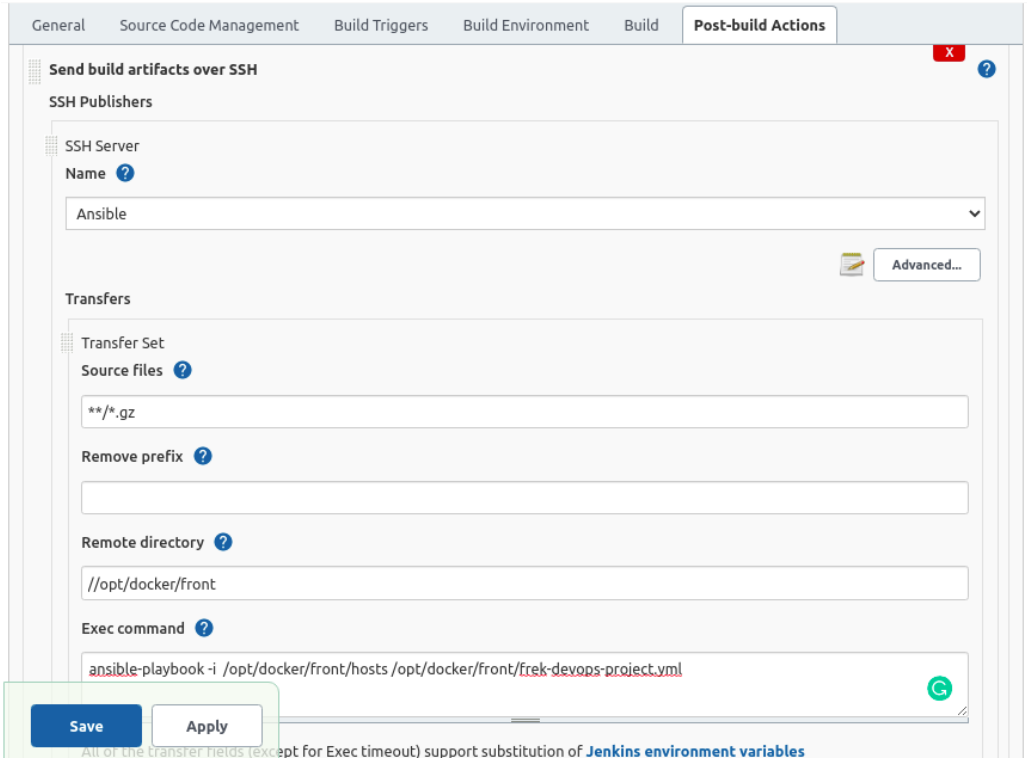


Figure 4.13 – Caractéristiques du job final

Ce « job » se termine par un succès :

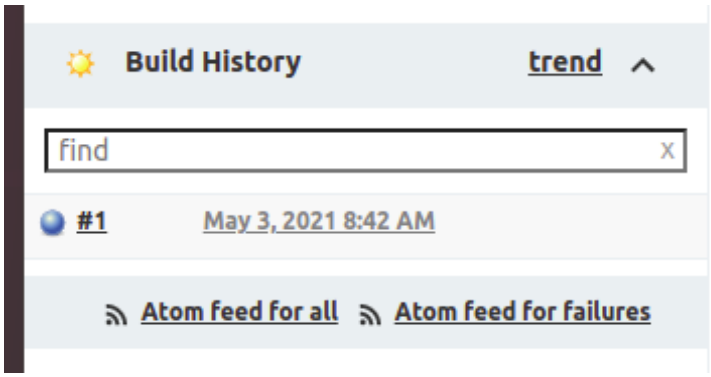


Figure 4.14 – Résultat d'exécution du test final

Nous remarquons la bulle bleue, ce qui explique que la construction de ce test est faite avec succès.
À ce niveau si nous voulons changer le code source il y aura le redéploiement automatique.

4.3.8 Intégration de cluster Kubernetes dans le pipeline

4.3.8.1 La mise en place d'un environnement Kubernetes

— Dans cette phase nous allons créer tout d'abord 3 instances Ubuntu 18.04 pour notre cluster kubernetes. Une instance pour le master node et les deux autres pour les slaves nodes.
— Installation de kubeadm, kubelet, kubectl, kubernetes-cni dans le master node ainsi que dans les slaves nodes.

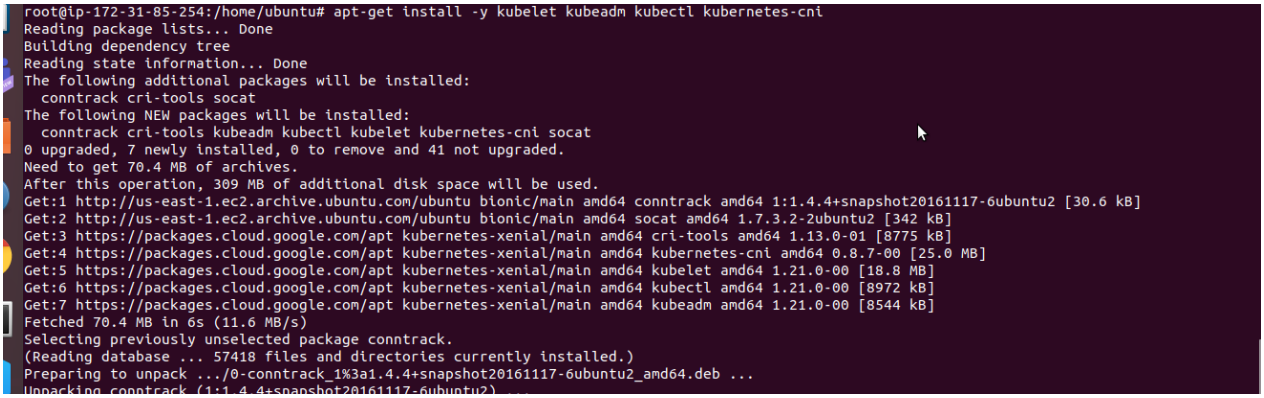


Figure 4.15 – Installation de kubeadm, kubelet, kubectl et kubernetes-cni

— L'étape suivante est la création des « AMI » pour le master node en utilisant la commande **kubeadm init**. Et ceci est le résultat de cette commande :

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 172.31.85.254:6443 --token vhqbv4.bgl9gd7440qp0s7m \
        --discovery-token-ca-cert-hash sha256:d90297ac307f2e9c4a57c9466d9b2d3b402688272b0baac50a0c7621fe33b598
root@ip-172-31-85-254:/home/ubuntu#
```

Figure 4.16 – kubeadm init

— Nous pouvons maintenant vérifier les nœuds qui se trouve dans notre serveur master.

```
root@ip-172-31-55-74:/home/ubuntu# kubectl get pods --all-namespaces
NAMESPACE     NAME                                       READY   STATUS    RESTARTS   AGE
kube-system   coredns-558bd4d5db-7h446                   0/1     Pending   0          114s
kube-system   coredns-558bd4d5db-qpsfn                   0/1     Pending   0          114s
kube-system   etcd-ip-172-31-55-74                       1/1     Running   0          2m2s
kube-system   kube-apiserver-ip-172-31-55-74             1/1     Running   0          2m2s
kube-system   kube-controller-manager-ip-172-31-55-74    1/1     Running   0          2m2s
kube-system   kube-proxy-wkt8n                           1/1     Running   0          114s
kube-system   kube-scheduler-ip-172-31-55-74             1/1     Running   0          2m1s
root@ip-172-31-55-74:/home/ubuntu# sudo su -
```

Figure 4.17 – Afficher les nœuds avec kubectl

4.3.8.2 La mise en place des nœuds esclaves (slaves nodes)

Dans ce stade nous allons intégrer le premier puis le deuxième nœud esclave avec le nœud principal ou encore appelé « master node », en utilisant la commande **join**.

```
ubuntu@ip-172-31-95-54:~$ sudo su -
root@k8s-slave1:~# kubeadm join 172.31.85.254:6443 --token vhqbv4.bgl9gd7440qp0s7m \
> --discovery-token-ca-cert-hash sha256:d90297ac307f2e9c4a57c9466d9b2d3b402688272b0baac50a0c7621fe33b598
[preflight] Running pre-flight checks
        [WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the Docker cgroup driver. The recommended driver is "systemd". Please follow th
e guide at https://kubernetes.io/docs/setup/cri/
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@k8s-slave1:~#
```

Figure 4.18 – Mettre en place le premier nœud

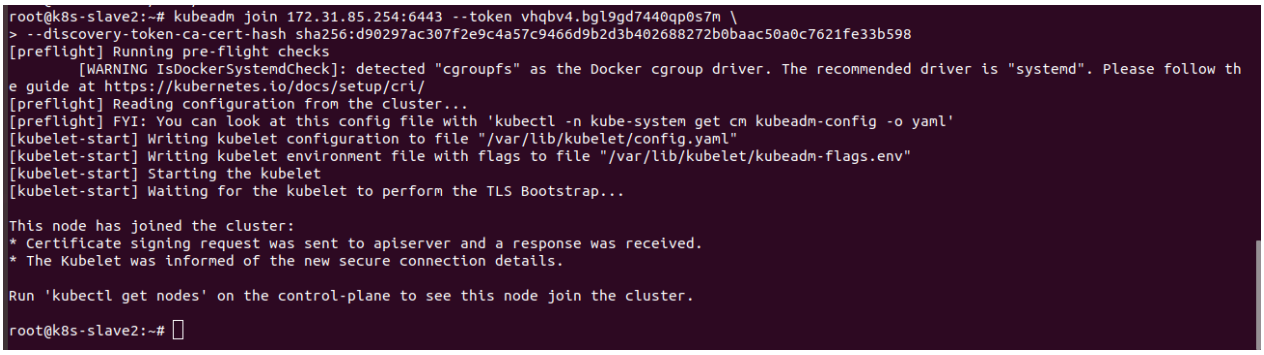


Figure 4.19 – Mettre en place le deuxième nœud

Maintenant nous vérifions que les deux nœuds sont bien mis en place et qui sont prêts pour l'utilisation.

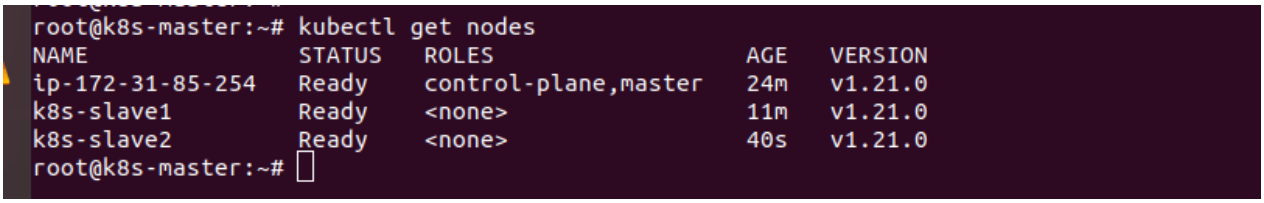


Figure 4.20 – kubectl get nodes

La figure 4.20 indique que les deux nœuds sont bien intégrés avec le nœud principal et ils sont « Ready » à utiliser.

4.3.8.3 Création des fichiers YAML de déploiement et de service

Dans cette phase, il s'agit d'écrire deux scripts, l'un est de type déploiement et l'autre de type service pour l'accès utilisateur.
En fait dans Kubernetes, un déploiement est une méthode de lancement d'un pod avec des applications conteneurisées et de s'assurer que le nombre nécessaire de réplicas est toujours en cours d'exécution sur le cluster.
D'autre part, un service est chargé d'exposer une interface à ces pods, ce qui permet l'accès au réseau depuis l'intérieur du cluster ou entre des processus externes et le service.

```
root@k8s-master:~# cat devops-deploy.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: devops-deployment
spec:
  selector:
    matchLabels:
      app: bouhnich-devops-project
  replicas: 1 # tells deployment to run 1 pods matching the template
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1

  template:
    metadata:
      labels:
        app: bouhnich-devops-project
    spec:
      containers:
      - name: bouhnich-devops-project
        image: bouhnich/frontend-devops-image
        imagePullPolicy: Always
        ports:
        - containerPort: 3000
root@k8s-master:~#
```

Figure 4.21 – Script YAML de déploiement

La figure 4.21 représente le fichier YAML de déploiement dans lequel nous avons précisé plusieurs paramètres :

- **Le champ Kind :** Préciser le type de fichier qui est dans notre cas déploiement.
- **Le champ metadata : name :** Donner un nom à notre déploiement.
- **Le champ selector :** Définit comment le déploiement trouve les pods à gérer.
- **Le champ replicas :** Dans ce champ nous précisons le nombre de pods répliqués.
- **Le champ template :** Dans ce champ nous précisons l'application sur laquelle se fait notre déploiement. Et en précisant le nom de conteneur et l'image docker utilisé ainsi que le numéro de port sur lequel le conteneur s'exécute

(31200).

```
root@k8s-master:~# cat devops-service.yml
apiVersion: v1
kind: Service
metadata:
  name: devops-service
  labels:
    app: bouhnich-devops-project
spec:
  selector:
    app: bouhnich-devops-project
  type: LoadBalancer
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 31200

root@k8s-master:~#
```

Figure 4.22 – Script YAML de service

Dans le fichier YAML représenté par la figure 4.22 nous avons écrit un Service sous le nom « bouhnich-service » pour l'app « bouhnich-devops-project » ainsi que le numéro de port sur lequel l'application s'exécute sur n'importe quel pod.

Donc après la création de deux scripts YAML nous devons les exécuter et vérifier qui sont bien créés.

```
root@k8s-cluster:~# kubectl get deploy
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
backend-deployment   2/2     2            2           131m
devops-deployment    2/2     2            2           93s
root@k8s-cluster:~# kubectl get services
NAME              TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
backend-service   LoadBalancer   10.96.72.50      <pending>     1337:31201/TCP   130m
devops-service    LoadBalancer   10.110.154.226   <pending>     3000:31200/TCP   89s
kubernetes        ClusterIP      10.96.0.1        <none>        443/TCP          2d2h
root@k8s-cluster:~#
```

Figure 4.23 – Vérification après exécution de deploy et service

Remarque : Nous avons créé de la même manière un autre script deploy et un autre script service pour la partie backend de notre application.

4.3.9 Intégration de Kubernetes avec Ansible

— Pour assurer la communication entre les deux serveurs nous effectuons l'échange de clés et l'authentification SSH entre eux.

— Puis nous créons un dossier Kubernetes sous le chemin /opt/ dans le serveur Ansible où nous allons créer les playbooks ansible à exécuter.

— Ensuite nous ajoutons l'adresse du serveur principal de kubernetes au fichier hosts.

```
[ansadmin@Ansible opt]$ sudo mkdir Kubernetes
[ansadmin@Ansible opt]$ ls
aws  containerd  docker  kubernetes
[ansadmin@Ansible opt]$
```

```
[ansadmin@Ansible kubernetes]$ cat hosts
[ansible-server]
localhost

[kubernetes]
54.87.160.120

[ansadmin@Ansible kubernetes]$
```

Figure 4.24 – Fichier Hosts dans /opt/Kubernetes/

— L'étape suivante est d'écrire un premier playbook pour l'exécution du fichier de déploiement qui se trouve dans l'instance du nœud principal.

```
[ansadmin@ip-172-31-43-227 kubernetes]$ cat kubernetes-bouhnich-deployment.yml
---
- name: Create pods using deployment
  hosts: kubernetes
#  become: true
  user: root

  tasks:
  - name: create a deployment
    command: kubectl apply -f devops-deploy.yml
```

Figure 4.25 – Playbook de déploiement

Et nous exécutons par la suite ce playbook :

```
[ansadmin@ip-172-31-43-227 kubernetes]$ ansible-playbook -i hosts kubernetes-bouhnich-deployment.yml
[WARNING]: Invalid characters were found in group names but not replaced, use
-vvvv to see details

PLAY [Create pods using deployment] ********************************************

TASK [Gathering Facts] *********************************************************
ok: [54.87.160.120]

TASK [create a deployment] *****************************************************
changed: [54.87.160.120]

PLAY RECAP *********************************************************************
54.87.160.120              : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 4.26 – Exécution du premier playbook

— Nous allons créer le deuxième playbook qui va réagir avec «bouhnich-service.yml» qui se trouve dans l'instance du nœud principale K8s_master.

```
[ansadmin@ip-172-31-43-227 kubernetes]$ cat kubernetes-bouhnich-service.yml
---
- name: create service for deployment
  hosts: kubernetes
  become: true
  user: root

  tasks:
  - name: create a service
    command: kubectl apply -f devops-service.yml

[ansadmin@ip-172-31-43-227 kubernetes]$
```

Figure 4.27 – Playbook de service

Et nous l'exécutons par la suite :

```
[ansadmin@ip-172-31-43-227 kubernetes]$ ansible-playbook -i hosts kubernetes-bouhnich-service.yml
[WARNING]: Invalid characters were found in group names but not replaced, use
-vvvv to see details

PLAY [create service for deployment] *******************************************

TASK [Gathering Facts] *********************************************************
ok: [54.87.160.120]

TASK [create a service] ********************************************************
changed: [54.87.160.120]

PLAY RECAP *********************************************************************
54.87.160.120              : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

[ansadmin@ip-172-31-43-227 kubernetes]$
```

Figure 4.28 – Exécution du deuxième playbook

Dans la partie suivante, nous allons créer 2 « jobs » dans Jenkins qui permettent de résumer tout ce travail d'une façon plus organisée et simple pour l'utilisateur.

4.3.10 CI Job dans Jenkins pour créer une image Docker dans le cluster kubernetes

Dans cette partie, nous allons créer un « job » CI qui assure la création d'une image docker localement dans notre serveur et le déploiement par la suite dans DockerHub.

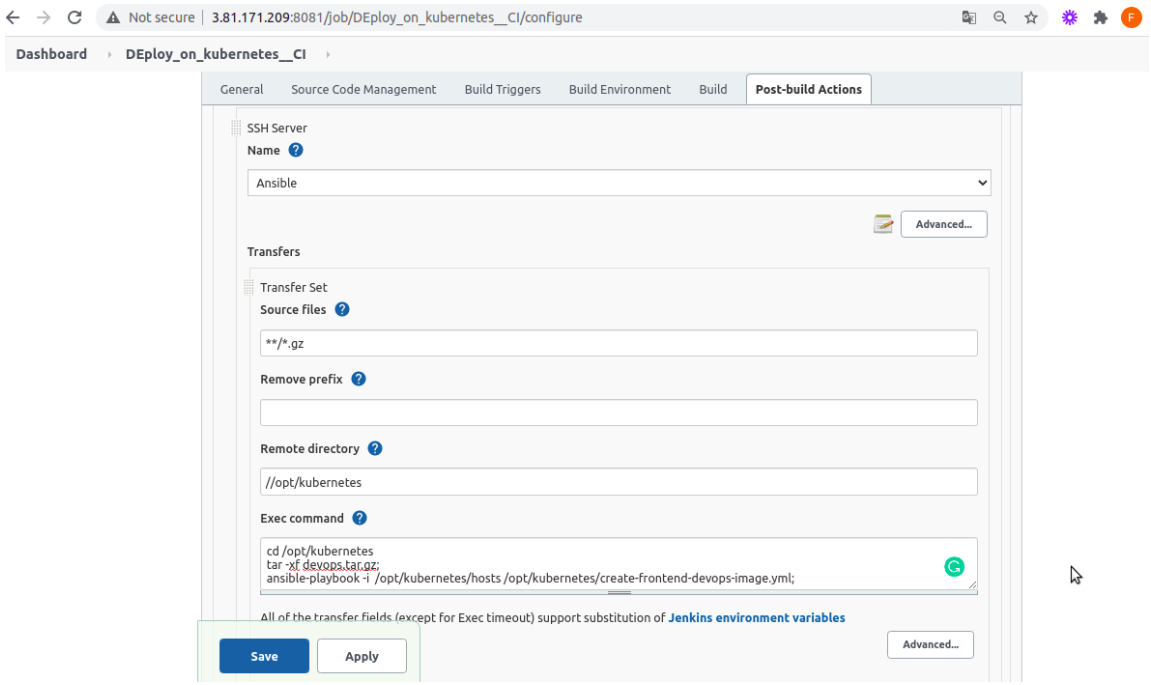


FIGURE 4.29 – CI job Kubernetes

Nous allons ajouter une nouvelle stratégie pour mieux organiser notre travail et le rendre simple à manipuler : si ce « job » est valide, il déclenche un autre « job » qui est le CD « job » d'une manière **automatique**

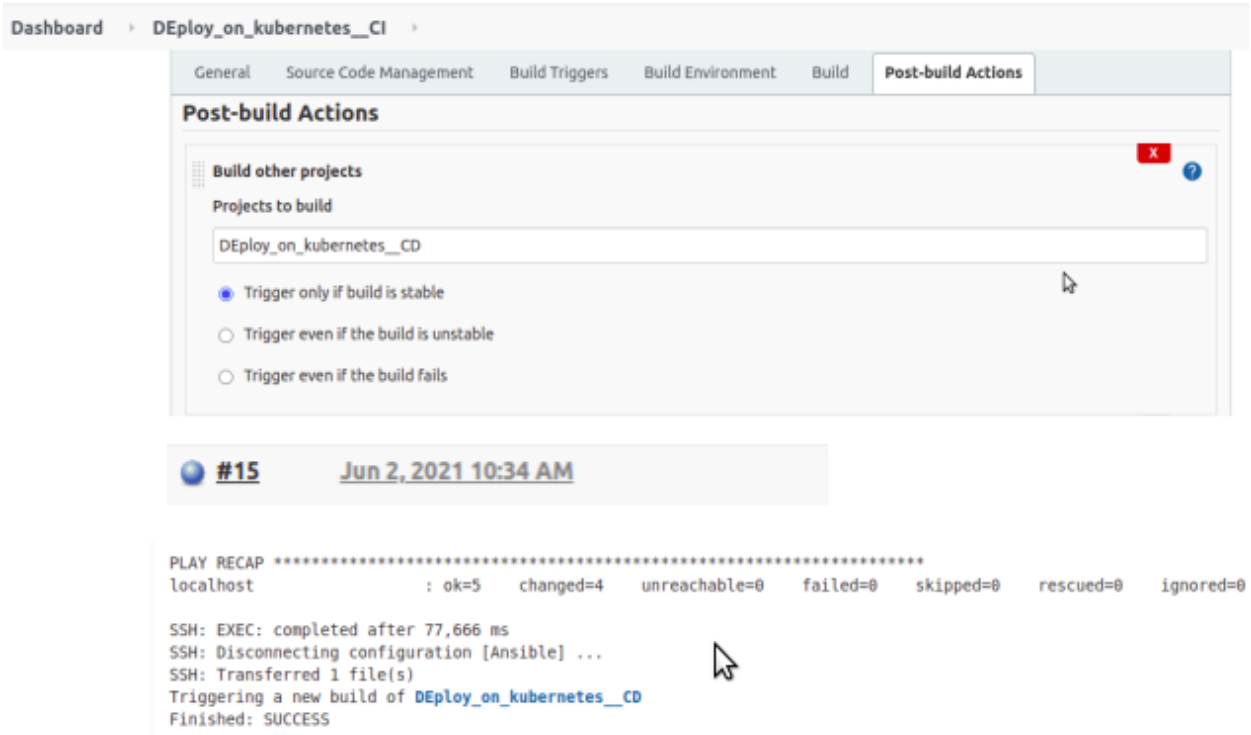


Figure 4.30 – Résultat CI job

Et donc dès que le CI « job » se termine avec succès le CD « job » sera lancer automatiquement :

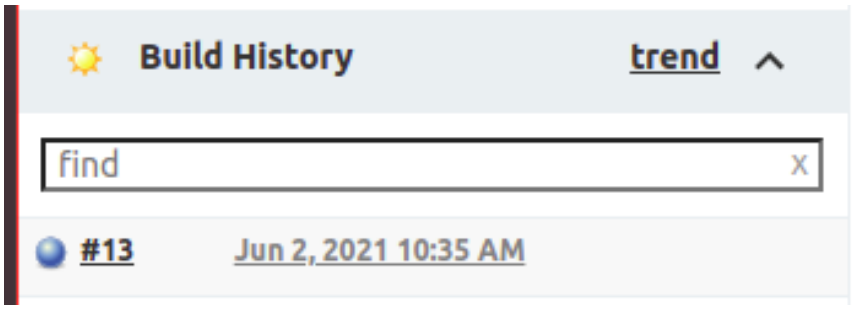


Figure 4.31 – Exécution automatique du CD job

4.3.11 CD Job dans Jenkins pour déployer l'image docker dans Kubernetes

Dans cette phase nous allons créer le deuxième « job » CD pour le déploiement dans nos nœuds Kubernetes en utilisant les 2 ansible Playbooks qui sont déjà créés

dans la partie précédente.

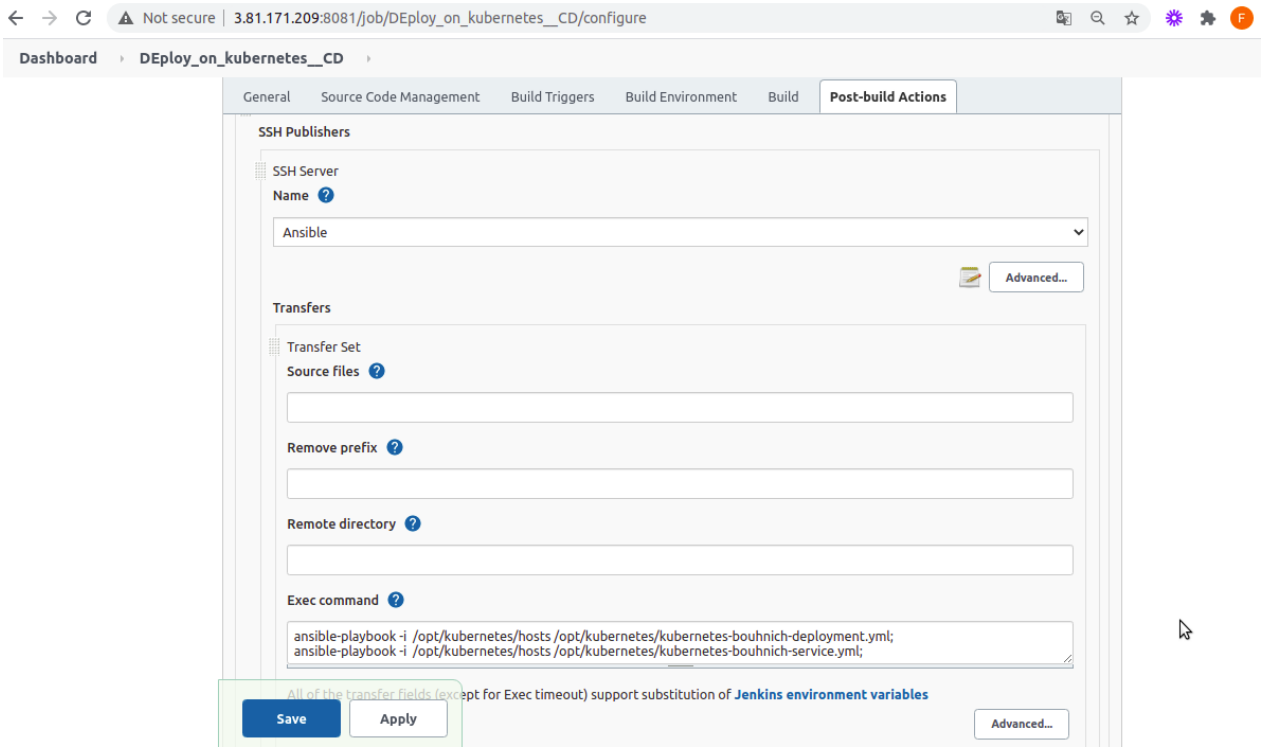


Figure 4.32 – CD job k8s

Résultat d'exécution de CD « job » :

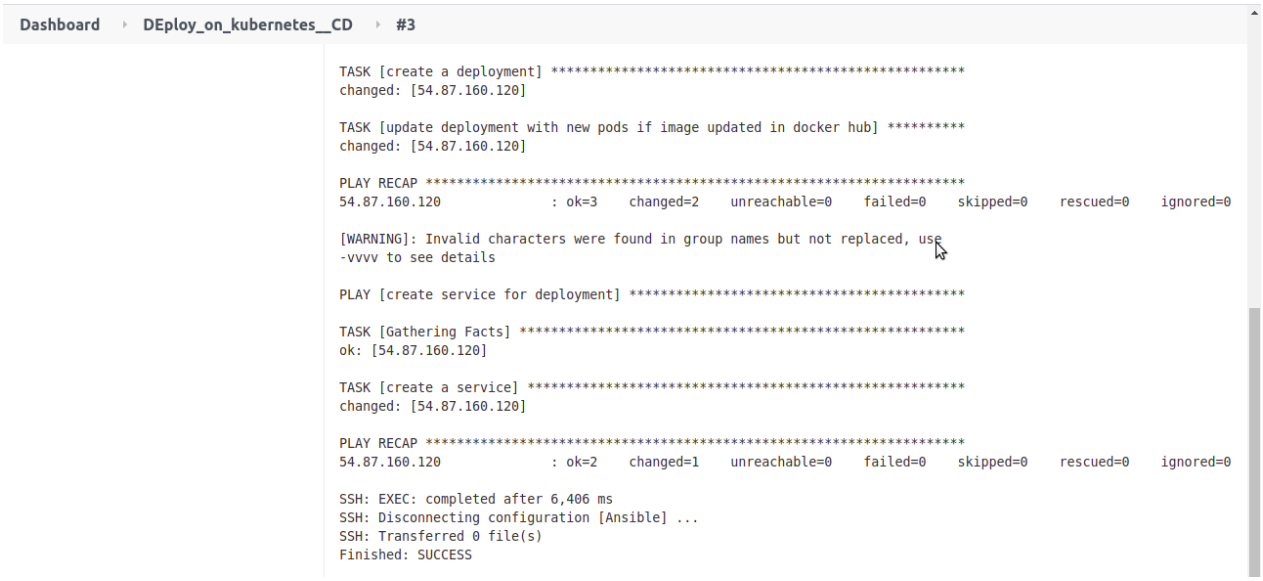


Figure 4.33 – Résultat CD job

4.3.12 Intégration de ESLint avec Jenkins

Eslint est outil d'analyse de code statique utilisé dans le développement de logiciels pour vérifier si le code source est conforme aux règles de codage.
Pour faire l'intégration de Eslint avec Jenkins nous avons téléchargé le plugin "Warnings Next Generation".

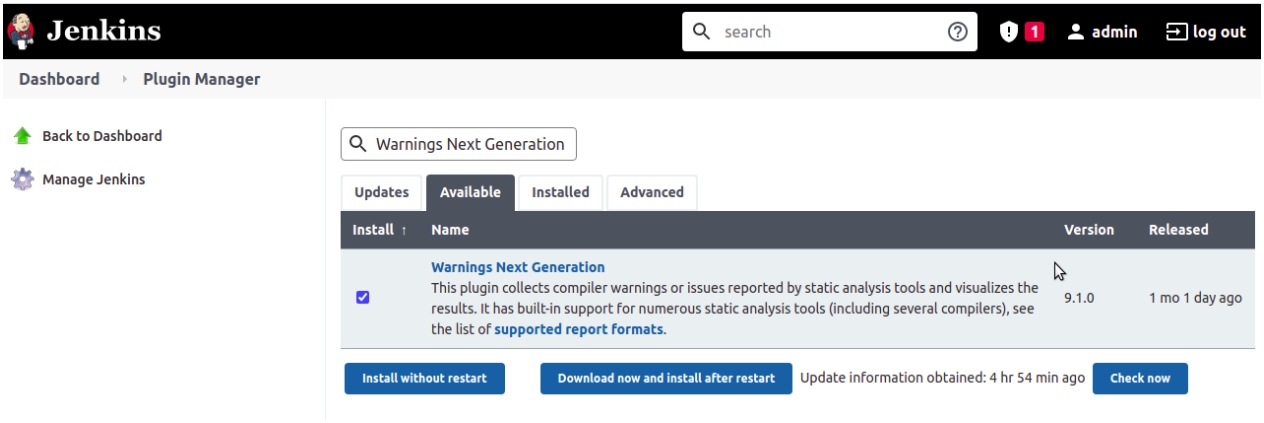


Figure 4.34 – Warnings Next Generation Plugin

Après nous configurons le « job » Jenkins Eslint. L'étape recordIssues recherche les problèmes dans un ensemble donné de fichiers et signale ces problèmes dans notre build. Cela activera un menu ESlint Warnings sur notre build.

4.3.13 Intégration de SonarQube dans le pipeline

— Nous lançons une nouvelle instance SonarQube dans AWS sur laquel nous installons sonarQube.

```
[root@ip-172-31-71-27 opt]# wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-6.7.7.zip
--2021-05-24 09:57:30--  https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-6.7.7.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 91.134.125.245
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|91.134.125.245|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 159921852 (153M) [application/zip]
Saving to: 'sonarqube-6.7.7.zip'

sonarqube-6.7.7.zip 100%[===================>] 152.51M  16.5MB/s    in 9.8s

2021-05-24 09:57:41 (15.5 MB/s) - 'sonarqube-6.7.7.zip' saved [159921852/159921852]

[root@ip-172-31-71-27 opt]# ls
sonarqube-6.7.7.zip
[root@ip-172-31-71-27 opt]# unzip sonarqube-6.7.7.zip
```

Figure 4.35 – Installation de SonarQube

— Nous configurons la base de données dans cette instance et nous l'administrons avec PostgreSQL.
— Nous installons sonar scanner dans Jenkins ainsi que le plugin « SonarQube Scanner ».

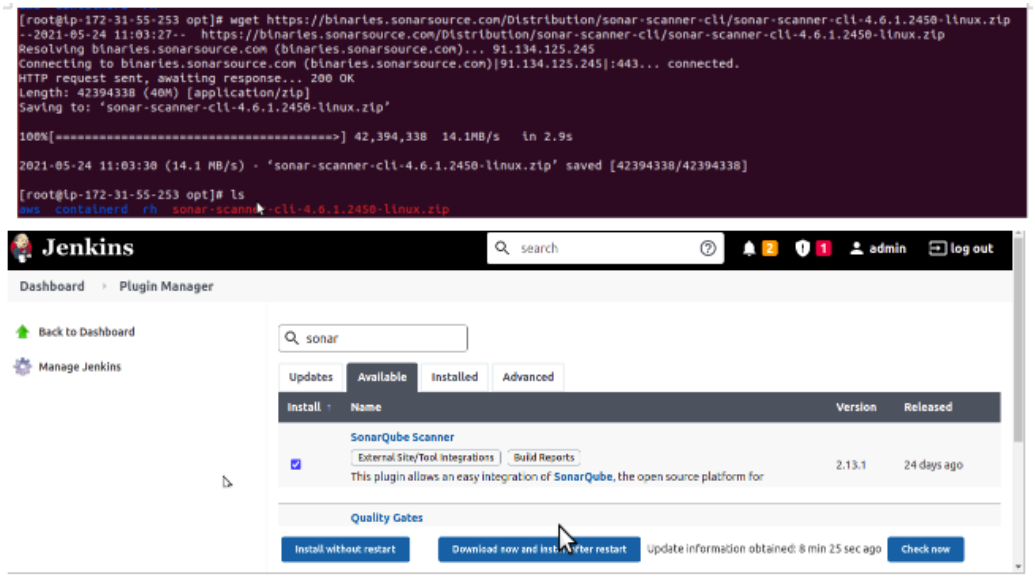


FIGURE 4.36 – Installation de Sonar Scanner

— Après la terminaison de l'installation nous configurons Sonar scanner pour assurer l'échange entre le serveur Jenkins et le serveur SonarQube.

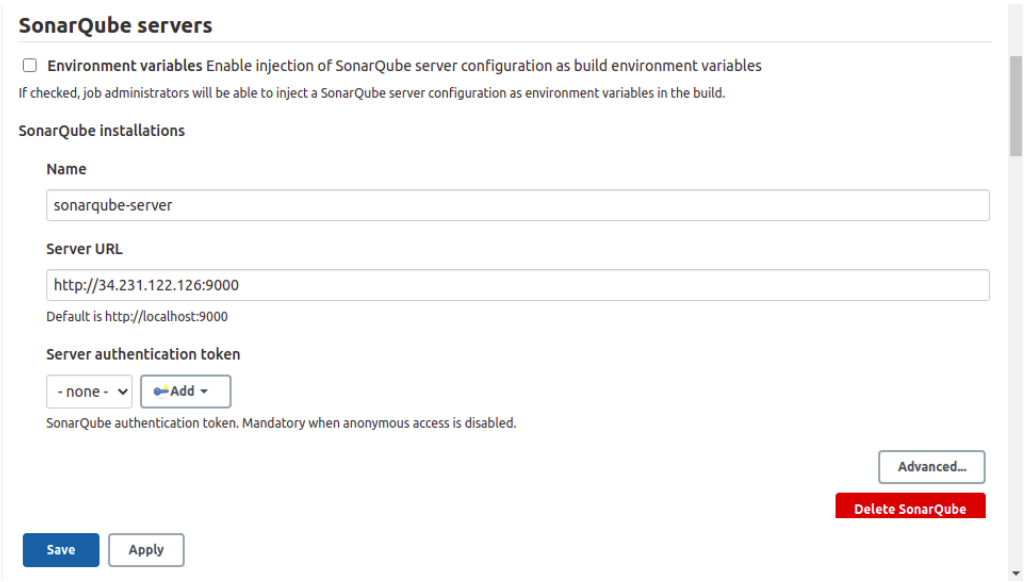


FIGURE 4.37 – Configuration de sonar scanner dans le serveur Jenkins

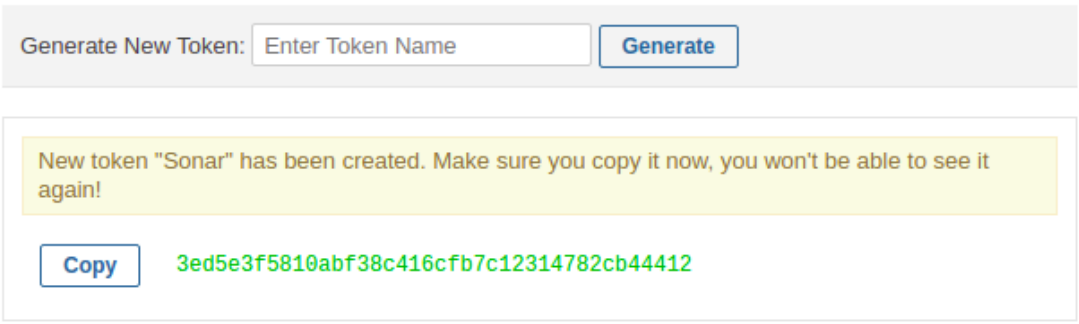


FIGURE 4.38 – Clé d'authentification entre SonarQube et Jenkins

— Maintenant nous créons un « job » jenkins et nous le lançons pour exécuter une analyse de contrôle qualité. ET nous remarquons que le « job » est terminé par un succès.

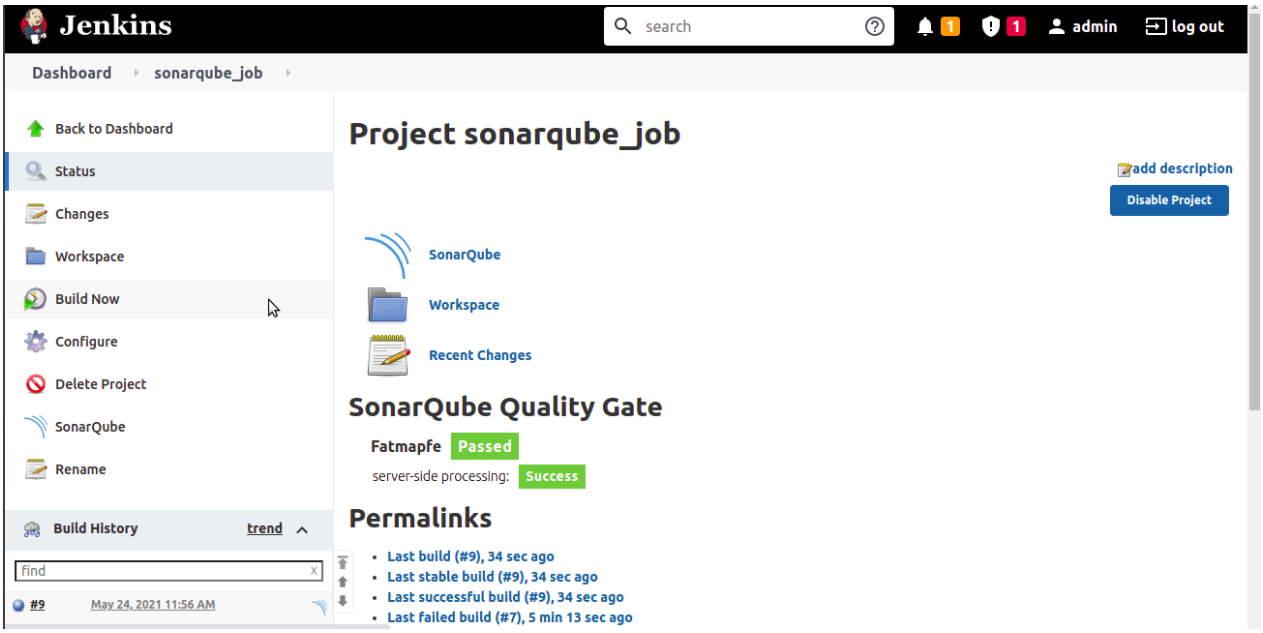


FIGURE 4.39 – Résultat de l'analyse sonar côté Jenkins

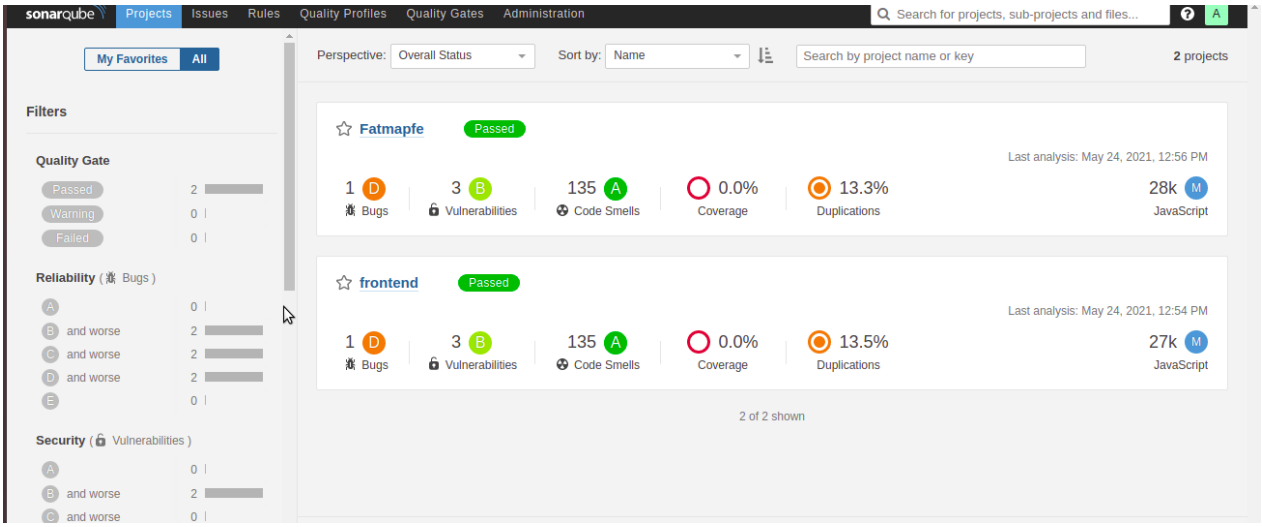


Figure 4.40 – Résultat de l'analyse côté Sonar

4.3.14 Partie monitoring avec Prometheus et Grafana

4.3.14.1 Mettre en place d'environnement Prometheus

- Nous créons une nouvelle instance Amazon Linux intitulée : Prometheus_server.
- Installation de Prometheus dans l'instance AWS en utilisant l'image docker de prometheus : # docker run -d –name prometheus -p 9090 :9090 prom/prometheus
- Initialement et comme preuve de concept, nous pouvons configurer Prometheus pour qu'il se surveille lui-même.

```
root@ip-172-31-25-158:/var/lib/docker/volumes/prometheus_config/_data
File  Edit  View  Search  Terminal  Help
# Load rules once and periodically evaluate them according to the global 'evalua
tion_interval'.
rule_files:
  # - "first_rules.yml"
  # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped
from this config.
  - job_name: 'prometheus'

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
    - targets: ['localhost:9090']
```

Figure 4.41 – Fichier de Configuration Prometheus

4.3.14.2 Intégration de Prometheus avec Jenkins

— Configuration de Prometheus pour récupérer les métriques de Jenkins. Pour faire ceci, nous devons éditer le fichier de configuration "prometheus.yml" en ajoutant les lignes de commandes présenter dans la figure suivante.

```
- job_name: 'jenkins'
  metrics_path: /prometheus
  static_configs:
    - targets: ['100.24.25.228:8080']
```

FIGURE 4.42 – Fichier de Configuration Prometheus

— Installation de plugin nécessaire dans Jenkins : Prometheus metrics plugin, qui va exposer les métriques Jenkins dans Prometheus.

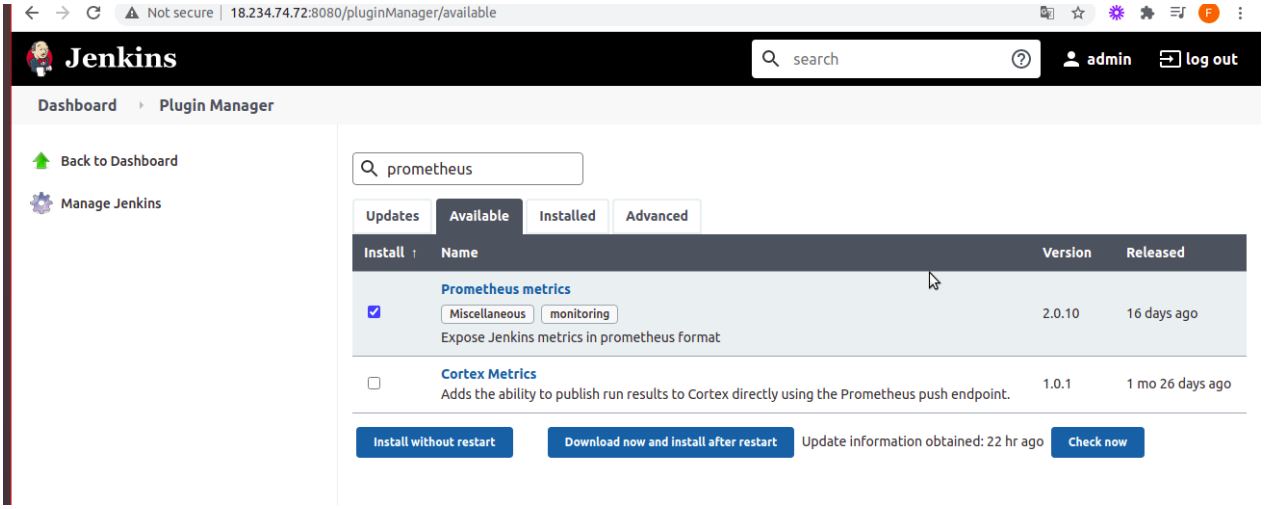


FIGURE 4.43 – Prometheus metrics plugin

— Après la terminaison de l'installation de plugin nous configurons Prometheus metrics pour assurer l'échange entre le serveur Jenkins et le serveur Prometheus.

4.3.14.3 Visualisation & Dashboards

— Création d'un conteneur Grafana en utilisant l'image docker publier dans DockerHub :
docker run -d –name grafana -p 3000 :3000 grafana/ grafana

```
[root@ip-172-31-25-158 ~]# docker ps -a
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS         PORTS                    NAMES
3642bae9b6cf   prom/prometheus   "/bin/prometheus --c…"   29 hours ago   Up 4 minutes   0.0.0.0:9090->9090/tcp   prometheus
78330c2ef7b6   grafana/grafana   "/run.sh"                36 hours ago   Up 2 seconds   0.0.0.0:3000->3000/tcp   grafana
[root@ip-172-31-25-158 ~]#
```

Figure 4.44 – Liste de conteneurs

— Connexion Prometheus à Grafana : Nous connectons à Grafana en accédant au serveur Grafana <serveur_ip@ :3000>.
— Puis nous ajoutons une source de données du type Prometheus comme l'indique la figure suivante.

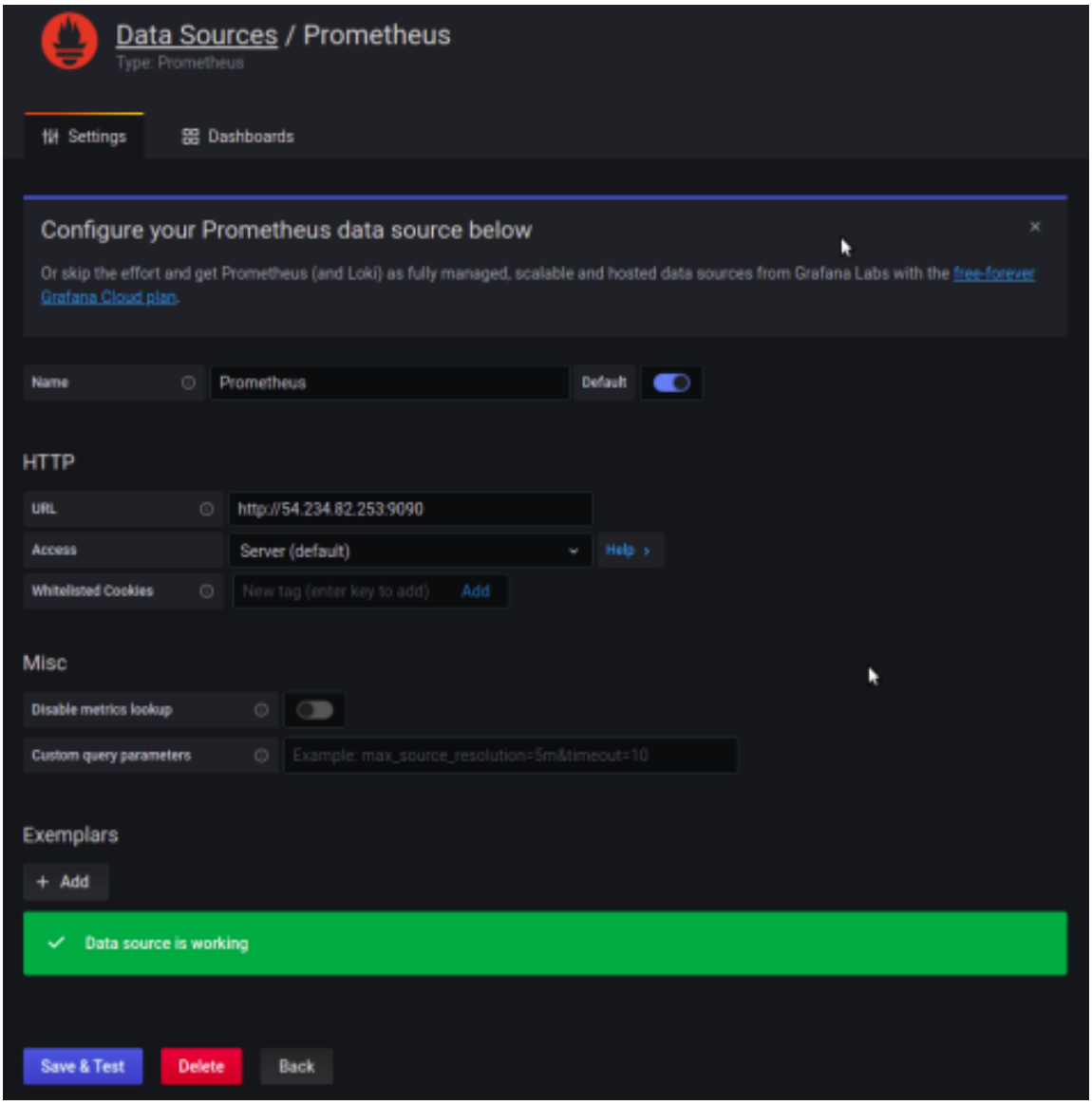


Figure 4.45 – Création d'une source de données du type Prometheus

— Créez un tableau de bord dans Grafana pour afficher les données. Nous devons maintenant créer un Dashbord pour visualiser les activités et les données de Jenkins dans Grafana.

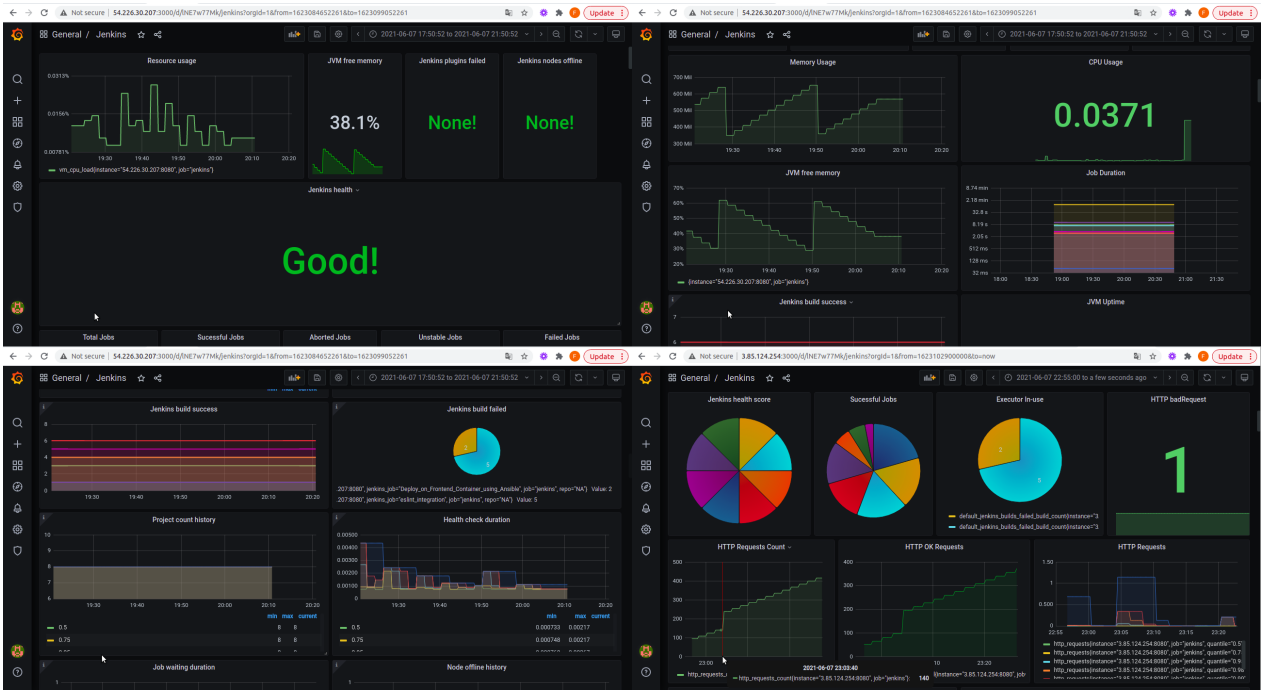


FIGURE 4.46 – Dashbord

— Nous avons maintenant un tableau de bord avec plusieurs panneaux qui représente une information sur Jenkins. Nous pouvons ajouter autant de panneaux que nous le souhaitons pour créer notre propre tableau de bord.
Dans ce tableau de bord, nous pouvons voir le nombre de build échoué, le nombre de build réussit pour chaque job, l'état de la dernière génération, la durée de jobs, taux d'utilisation de mémoire et CPU, nombre de requêtes HTTP, mauvaises requêtes HTTP et bonnes requêtes, etc.

4.4 Conclusion

Au cours de ce dernier chapitre, nous avons vu la dernière partie de ce projet qui est l'implémentation de notre solution. Nous avons donc vu comment mettre en place nos serveurs, ensuite nous avons présenté la partie déploiement d'application de deux manières ; l'une dans un serveur Docker et l'autre dans un cluster kubernetes, nous avons aussi expliqué nos scripts d'automatisation ainsi que les tâches dans Jenkins pour simplifier l'exécution de ces jobs. Enfin nous avons surveillé nos instances à l'aide de Prometheus et Grafana. Nous avons ainsi illustré les fonctionnalités implémentées à l'aide des captures d'écran.

À partir de ce moment, chaque application traverse un cycle de vie rapide et complet, de la phase d'intégration à la phase de déploiement facilitant automatiquement la production de cette application le plus tôt possible.

Conclusion Générale et Perspectives

Ce travail s'inscrit dans le cadre de la réalisation du projet de fin d'études, au sein de DigiCross, dans lequel nous étions amenés à mettre en place un pipeline d'intégration continue et de déploiement continue pour les applications web multi-services. Ce projet nous permet d'observer l'évolution de l'application tout au long de son cycle de vie de manière continue. Il nous permet ainsi de découvrir les erreurs d'une manière plus rapide afin d'y corriger dès qu'elles se produisent.

Afin d'atteindre les objectifs de ce projet, nous avons procédé par une suite d'étapes bien étudiées. En effet, il était nécessaire en premier lieu d'étudier le cadre général du projet. Cette étude a été très utile pour comprendre la problématique, faire une analyse de l'existant ainsi que ses limites, et proposer une solution adéquate à cette problématique. En second lieu, nous avons étudié les différentes notions et concepts nécessaires pour une compréhension approfondie de ce projet, telles que l'approche DevOps, l'intégration continue, le concept de la virtualisation ainsi que la notion du cloud computing. En particulier, le choix des outils est l'une des nombreuses solutions qui existent sur le marché, c'est pourquoi une étude comparative de ces outils est nécessaire à faire.

Puis, pour mettre en place ces outils, nous avons identifié les exigences fonctionnelles et non fonctionnelles et nous avons attaqué ainsi la partie conception de la solution proposée avec le formalisme UML qui a permis de bien modéliser notre système de manière à ce que la compréhension de celui-ci devient plus facile et que sa mise en place soit plus rapide. Finalement, nous avons mis en œuvre notre pipeline CI/CD ainsi que la partie monitoring de nos serveurs.

Néanmoins, dans ce travail, nous avons rencontré des problèmes liés aux ressources budgétaires sur le cloud AWS. En fait, nous étions obligés de réduire l'utilisation d'Amazon EC2 en arrêtant et en démarrant manuellement nos instances à des heures prédéfinies.

Ce stage nous a beaucoup apportés, tant en termes personnel que d'expertise. Il nous a permis de mieux comprendre et d'appréhender l'organisation d'une entreprise, ses contraintes et sa mission. Cela nous donne l'occasion de mieux comprendre le concept DevOps.

Nous proposons, comme perspective à ce travail, l'ajout d'une partie d'autoscaling. En fait l'autoscaling est une fonctionnalité de cloud computing qui permet de faire évoluer automatiquement les services cloud tels que les capacités des serveurs ou les machines virtuelles, en fonction de situations définies telles que le trafic ou les niveaux d'utilisation.

Annexes

— **Connexion SSH :** Nous connectons à nos instances AWS à traves le protocole SSH et une clé privée (.pem) qui est créée par AWS lorsque nous avons lancé une instance. Donc pour faire ceci nous exécutons une ligne de commande dans notre terminal :



Figure 4.47 – Connexion SSH

— **Connexion entre Jenkins et Git :** Pour que nous puissons récupérer le code source et assurer la communication avec Gitlab nous avons téléchargé les "plugins" nécessaires : "Git Plugin" et "Gitlab Plugin".

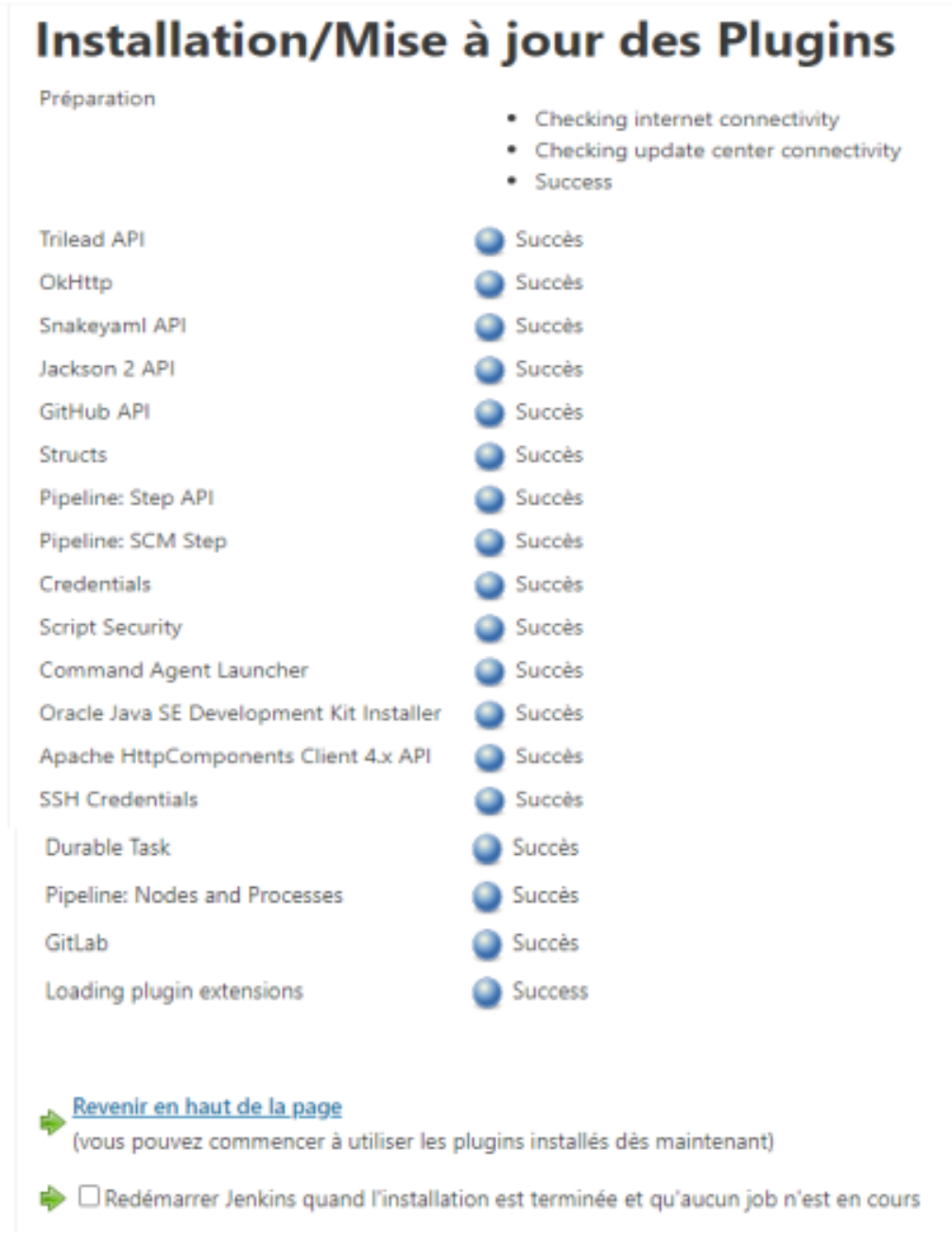


Figure 4.48 – Gitlab Plugin

— **Configuration de Jenkins pour les projets GitLab privés :** Dans notre projet nous avons travaillé avec un répertoire privé de Gitlab. Donc pour correctement intégrer le code du Gitlab avec Jenkins nous devons créé des paires de clés ssh (clé privée et clé publique) pour l'utilisateur de jenkins, en utilisant la commande : # ssh-keygen

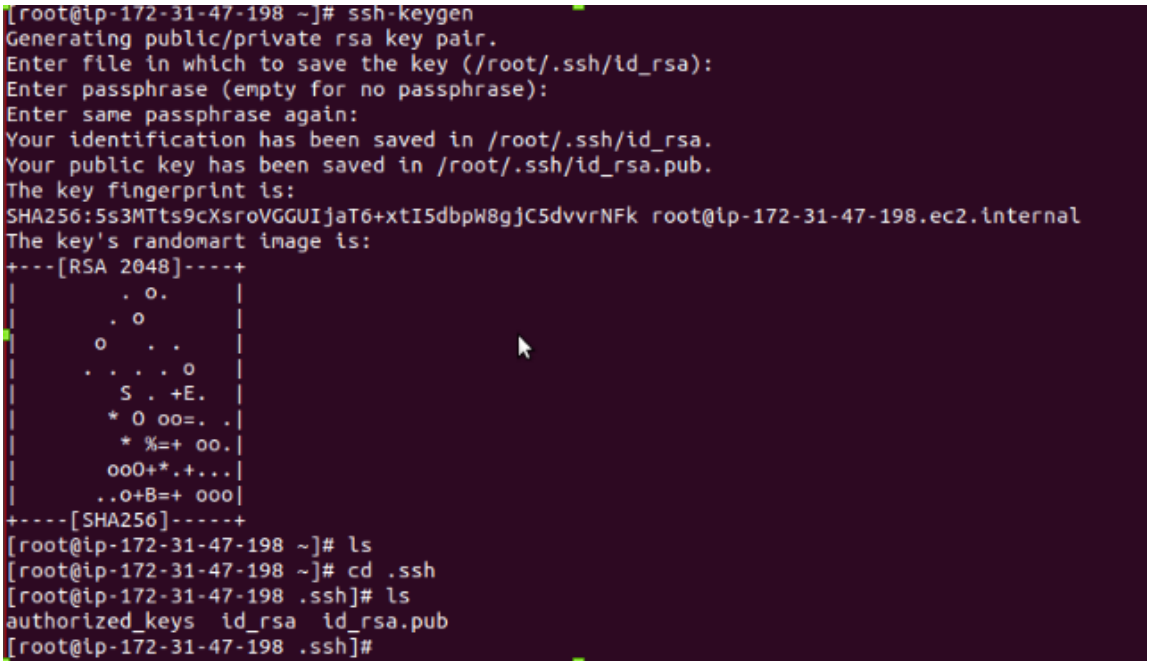


Figure 4.49 – Génération de paires de clés ssh

— Nous copions la clé publique « id_rsa.pub ».
Dans les paramètres du profil GitLab nous choisissons SSH Keys et nous ajoutons une clé ssh à GitLab en collant la clé publique générée dans le serveur jenkins, cela permettra à notre serveur Jenkins d'accéder à notre répertoire privé dans GitLab.

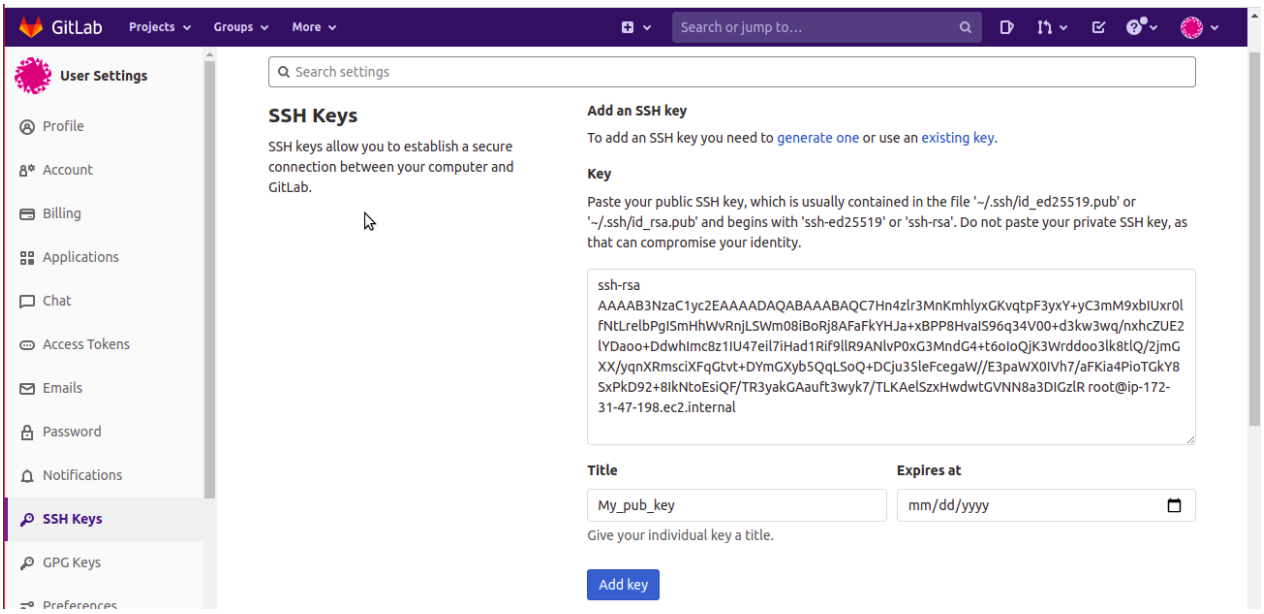


FIGURE 4.50 – Création d'une clé ssh à GitLab

— Puis dans le serveur jenkins nous créons une nouvelle connexion gitlab en utilisant la clé privée « id_rsa ».

— **Installation et démarrage des services docker :**

#yum install docker -y

#docker –version : pour vérifier la version de docker

#service docker start

Par défaut, la commande docker n'est exécutée que sous l'utilisateur root, et l'utilisateur normal n'a pas les autorisations nécessaires pour exécuter la commande docker jusqu'à ce qu'elle soit ajoutée au groupe docker via la commande suivante :

#usermod -aG docker dockeradmin

— **Démarrage de serveur SonarQube :**

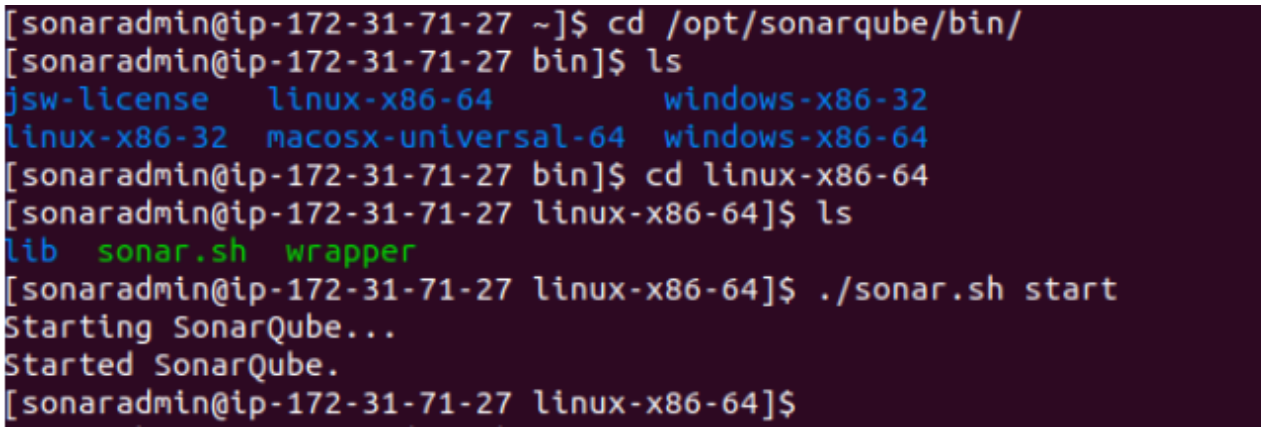


Figure 4.51 – Démarrage de service sonar

— **Configuration de serveur Ansible :**
En tant que bonne pratique de sécurité, il n'est pas recommandé d'exécuter le service Ansible en tant que root. Par conséquent, créez un nouvel utilisateur nommé ansadmin et accordez à sudo l'accès à la gestion des services ansible.
#useradd ansadmin
#passwd ansadmin
#echo "ansadmin ALL=(ALL) NOPASSWD : ALL" » /etc/sudoers

— **Docker compose file :**

```
[root@docker devops]# cat docker-compose.yml
version: '3'
services:
  strapi:
    container_name: strapi
    image: strapi/strapi
    environment:
      DATABASE_NAME: fretkk
      DATABASE_HOST: cluster0.tfgwh.mongodb.net
      DATABASE_PORT: 27017
      DATABASE_USERNAME: rimhorchani
      DATABASE_PASSWORD: Rimitta*0
    ports:
      - 1337:1337
    volumes:
      - ./backend:/srv/app

  frontend:
    build: ./frontend
    container_name: devops_container
    environment:
          - STRAPI_API_URL= http://172.31.63.235:1337
    ports:
         - 3000:3000
[root@docker devops]#
```

Figure 4.52 – Fichier Docker Compose

Liste des références

[1] Ansible. [En ligne]. Available : https://docs.ansible.com/.

[2] Architecture docker. [En ligne]. Available : https://geekflare.com/fr/docker-architecture/.

[3] Codacy. [En ligne]. Available : https://www.codacy.com/.

[4] Devops. [En ligne]. Available : https://iac.goffinet.org/infrastructure-as-code/introduction-devops/.

[5] Devops : implémentation d'un ci/cd. [En ligne]. Available : https://www.softfluent.fr/blog/devops-implementation-ci-cd/.

[6] Devops tool. [En ligne]. Available : https://www.cgtscorp.com/devops-tool/.

[7] Digicross. [En ligne]. Available : https://digicross.herokuapp.com/index.html?fbclid=IwAR3StnODg5Aam1QYFXJeFgQVeFMZUE4dH-b5r9c9QXQ1MuPPOgbKI6owpHE.

[8] Docker. [En ligne]. Available : https://docs.docker.com/get-started/overview/.

[9] Docker containers and vms. [En ligne]. Available : https://www.weave.works/blog/a-practical-guide-to-choosing-between-docker-containers-and-vms.

[10] En quoi consiste l'intégration continue. [En ligne]. Available : https://openclassrooms.com/en/courses/4087056-testez-et-suivez-letat-de-votre-application-php/4419471-en-quoi-consiste-lintegration-continue.

[11] Git. [En ligne]. Available : https://fr.wikipedia.org/wiki/Git.

[12] Grafana. [En ligne]. Available : https://www.metricfire.com/blog/what-is-grafana/.

[13] Infographic - what's the difference between continuous delivery and continuous deployment ? [En ligne]. Available : https://www.gocd.org/2017/10/17/difference-between-continuous-delivery-continuous-deployment-infographic/.

[14] Jenkins. [En ligne]. Available : https://www.jenkins.io/doc/.

[15] Kubernetes. [En ligne]. Available : https://geekflare.com/kubernetes-architecture/.

[16] L'intégration continue. [En ligne]. Available : http://igm.univ-mlv.fr/~dr/XPOSE2011/IntegrationContinue/pourquoi.html?fbclid=IwAR3x6HoVxUWi0WvP8FcAT0-h3ITP3paXTcQ_gYDbz1zEBb5-on8yrwIQvYE.

[17] Outils open source devops : lesquels déployer. [En ligne]. Available : https://www.syloe.com/outils-open-source-devops-lesquels-deployer/.

[18] Prometheus. [En ligne]. Available : https://prometheus.io/docs/introduction/overview/.

[19] Qu'est-ce que la livraison continue ? [En ligne]. Available : https://aws.amazon.com/fr/devops/continuous-delivery/.

[20] Scrutinizer. [En ligne]. Available : https://scrutinizer-ci.com/.

[21] Sonarqube. [En ligne]. Available : https://www.troispointzero.fr/le-blog/sonarqube/.

[22] CI. [En ligne]. Available : http://www-igm.univ-mlv.fr/~dr/XPOSE2012/Integration%20Continue/concept.html.

[23] Virtualisation. [En ligne]. Available : https://www.lebigdata.fr/virtualisation-definition.

[24] virtualisation. [En ligne]. Available : https://jeremygrondin.weebly.com/veille.html.

[25] La culture devops un nouveau mouvement. [En ligne], 2019-09-11. Available : http://thomasrossier.fr/papers/la-culture-devops-pourquoi-si-important.php.

[26] Breaking down silos between development operations. [En ligne], DECEMBER 20,2017. Available : https://www.accenture.com/us-en/blogs/software-engineering-blog/shinde-development-operations-silos.